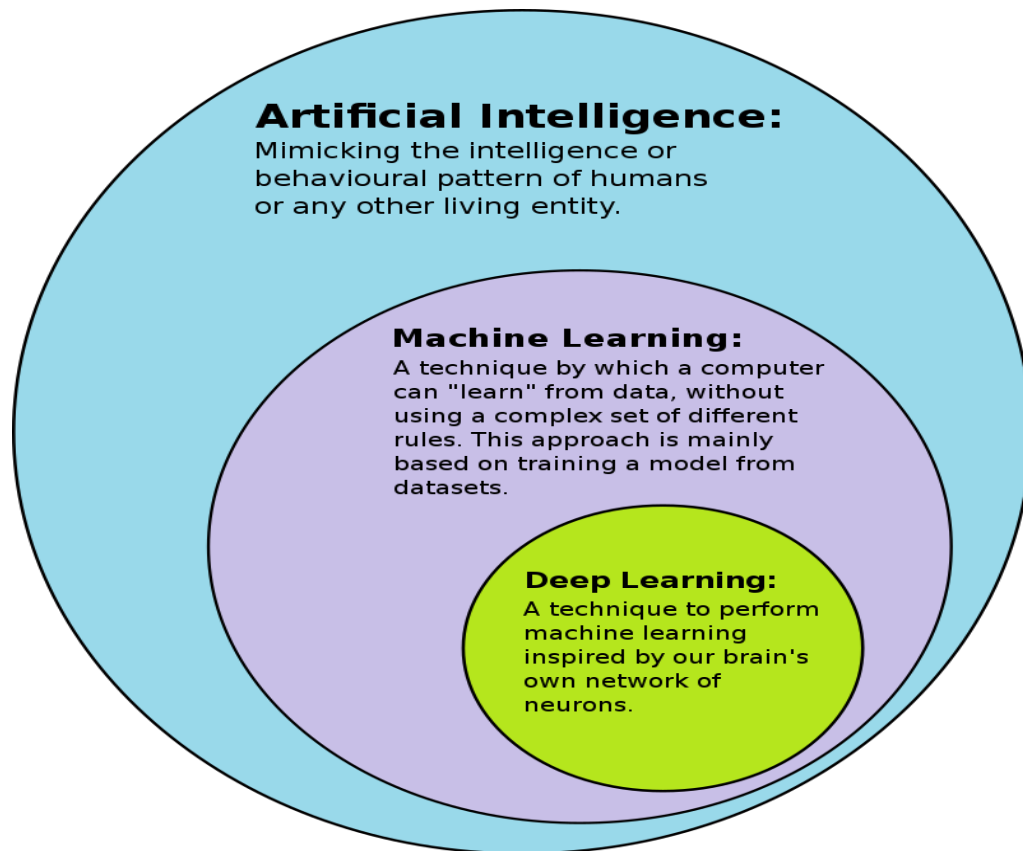


HISTORY OF DEEP LEARNING



- It's one kind of supervised machine learning, in which a computer is provided a training set of examples to learn a function, where each example is a pair of an input and an output from the function.
- The more complex the task, the bigger the training set has to be.
- Ex. Google's voice recognition algorithms
- Responsible for recent advances in computer vision, speech recognition, natural language processing, and audio recognition.

HISTORY OF DEEP LEARNING

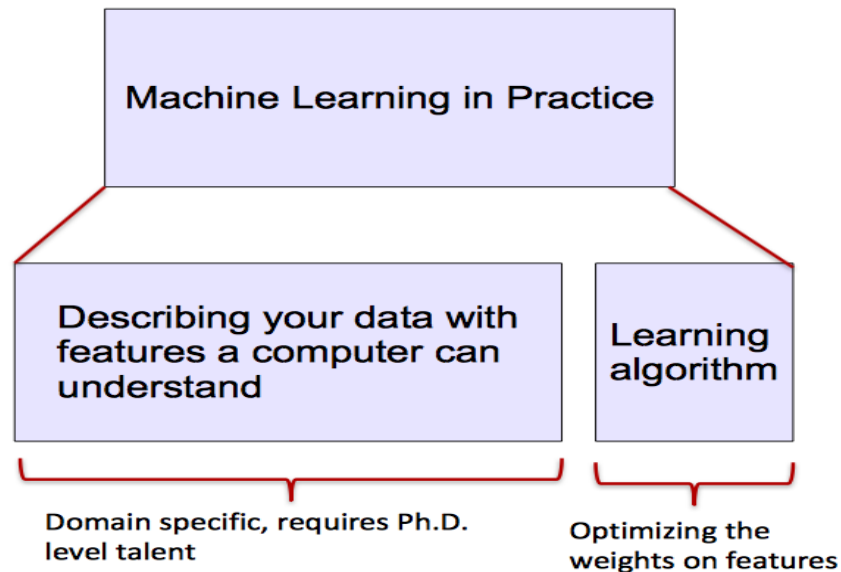
- 1989: Scientists were able to create algorithms that used deep neural networks, but training times for the systems were measured in days, making them impractical for real-world use.
- 1992: Juyang Weng publishes Cresceptron, a method for performing 3-D object recognition automatically from cluttered scenes.
- Mid-2000s: The term “deep learning” begins to gain popularity after a paper by Geoffrey Hinton and Ruslan Salakhutdinov showed how a many-layered neural network could be pre-trained one layer at a time.
- 2009: NIPS Workshop on Deep Learning for Speech Recognition discovers that with a large enough data set, the neural networks don’t need pre-training, and the error rates drop significantly.

HISTORY OF DEEP LEARNING

- 2012: Artificial pattern-recognition algorithms achieve human-level performance on certain tasks. And Google's deep learning algorithm discovers cats.
- 2014: Google buys UK artificial intelligence startup Deepmind for £400m
- 2015: Facebook puts deep learning technology – called DeepFace – into operations to automatically tag and identify Facebook users in photographs. Algorithms perform superior face recognition tasks using deep networks that take into account 120 million parameters.
- 2016: Google DeepMind's algorithm AlphaGo masters the art of the complex board game Go and beats the professional go player Lee Sedol at a highly publicised tournament in Seoul.

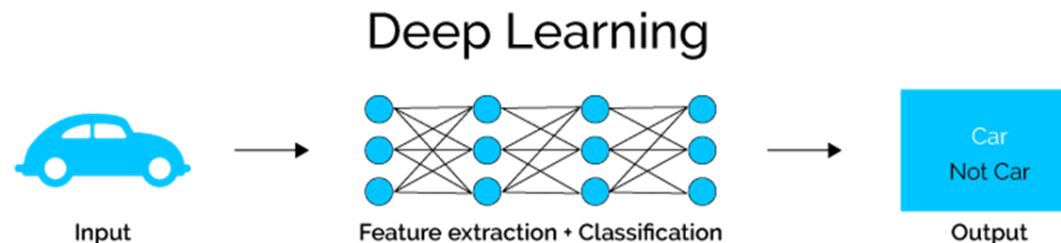
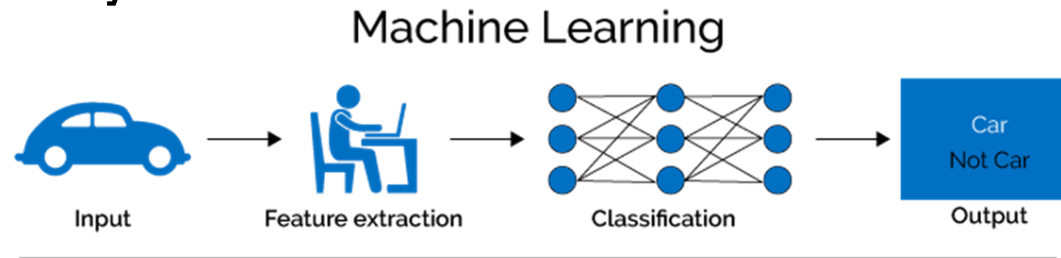
ML VS. DEEP LEARNING

- Most machine learning methods work well because of human-designed representations and input features
- ML becomes just optimizing weights to best make a final prediction



WHAT IS DEEP LEARNING?

- A machine learning subfield of learning representations of data. Exceptional effective at learning patterns.
- Deep learning algorithms attempt to learn (multiple levels of) representation by using a hierarchy of multiple layers
- If you provide the system tons of information, it begins to understand it and respond in useful ways.



WHY IS DL USEFUL?

- Manually designed features are often **over-specified**, **incomplete** and take a **long time to design** and validate
- Learned Features are **easy to adapt**, **fast** to learn
- Deep learning provides a very **flexible**, **universal**, learnable framework for representing world, visual and linguistic information.
- Can learn both unsupervised and supervised
- Utilize large amounts of training data

CHALLENGES IN DL

- Application: Image classification

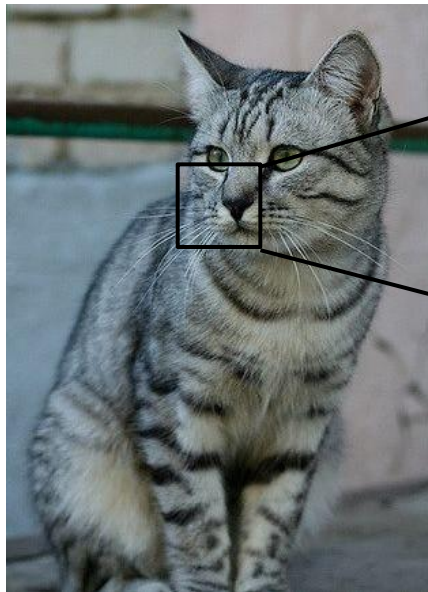


This image by Nikita is
licensed under [CC-BY 2.0](#)

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}

→ cat

THE PROBLEM



This image by Nikita is
licensed under CC-BY 2.0

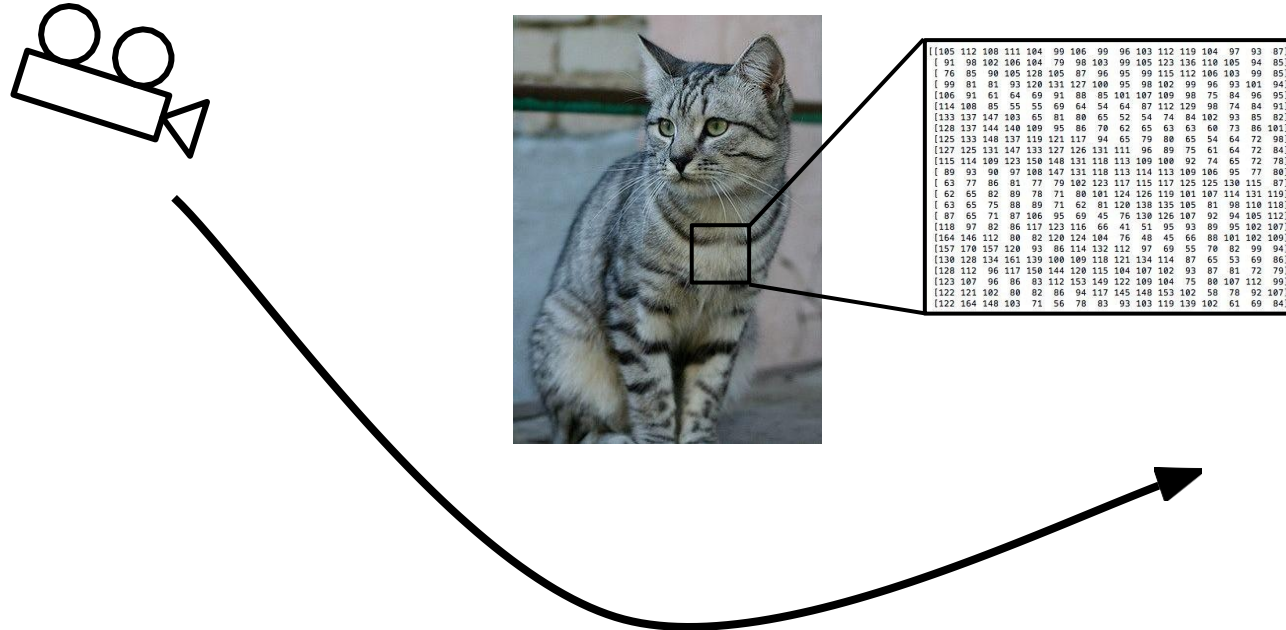
105	112	108	111	104	99	106	99	96	103	112	119	104	97	93	87]
91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
76	85	90	105	128	105	87	96	95	99	115	112	106	103	99	85]
99	81	81	93	120	131	127	100	95	98	102	99	96	93	101	94]
106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	95]
114	108	85	55	55	69	64	54	64	87	112	129	98	74	84	91]
133	137	147	103	65	81	80	65	52	54	74	84	102	93	85	82]
128	137	144	140	109	95	86	70	62	65	63	63	60	73	86	101]
125	133	148	137	119	121	117	94	65	79	80	65	54	64	72	98]
127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
115	114	109	123	150	148	131	118	113	109	100	92	74	65	72	78]
89	93	90	97	108	147	131	118	113	114	113	109	106	95	77	80]
63	77	86	81	77	79	102	123	117	115	117	125	125	130	115	87]
62	65	82	89	78	71	80	101	124	126	119	101	107	114	131	119]
63	65	75	88	89	71	62	81	120	138	135	105	81	98	110	118]
87	65	71	87	106	95	69	45	76	130	126	107	92	94	105	112]
118	97	82	86	117	123	116	66	41	51	95	93	89	95	102	107]
164	146	112	80	82	120	124	104	76	48	45	66	88	101	102	109]
157	170	157	120	93	86	114	132	112	97	69	55	70	82	99	94]
130	128	134	161	139	100	109	118	121	134	114	87	65	53	69	86]
128	112	96	117	150	144	120	115	104	107	102	93	87	81	72	79]
123	107	96	86	83	112	153	149	122	109	104	75	80	107	112	99]
122	121	102	80	82	86	94	117	145	148	153	102	58	78	92	107]
122	107	102	71	62	78	89	89	103	112	99	82	51	69	84]	]

What the computer sees

An image is just a big grid of
numbers between $[0, 255]$:
e.g. $800 \times 600 \times 3$
(3 channels RGB)

CHALLENGES

- Viewpoint orientation: All pixels change when the camera moves



All pixels change when
the camera moves!

CHALLENGES

- Illumination



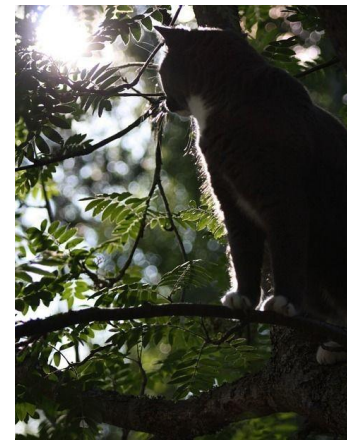
[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

CHALLENGES

- Deformation



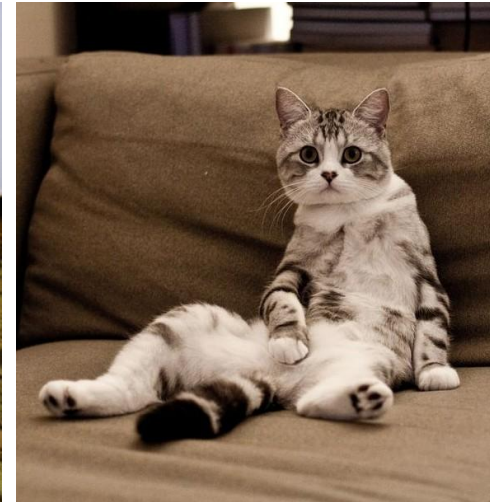
This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [sare bear](#) is licensed under [CC-BY 2.0](#)



This image by [Tom Thai](#) is licensed under [CC-BY 2.0](#)

CHALLENGES

- Occlusion



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) by [jonsson](#) is licensed under [CC-BY 2.0](#)

CHALLENGES

- Background clutter



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

CHALLENGES

- Intraclass Variation



This image is [CC0 1.0](#) public domain

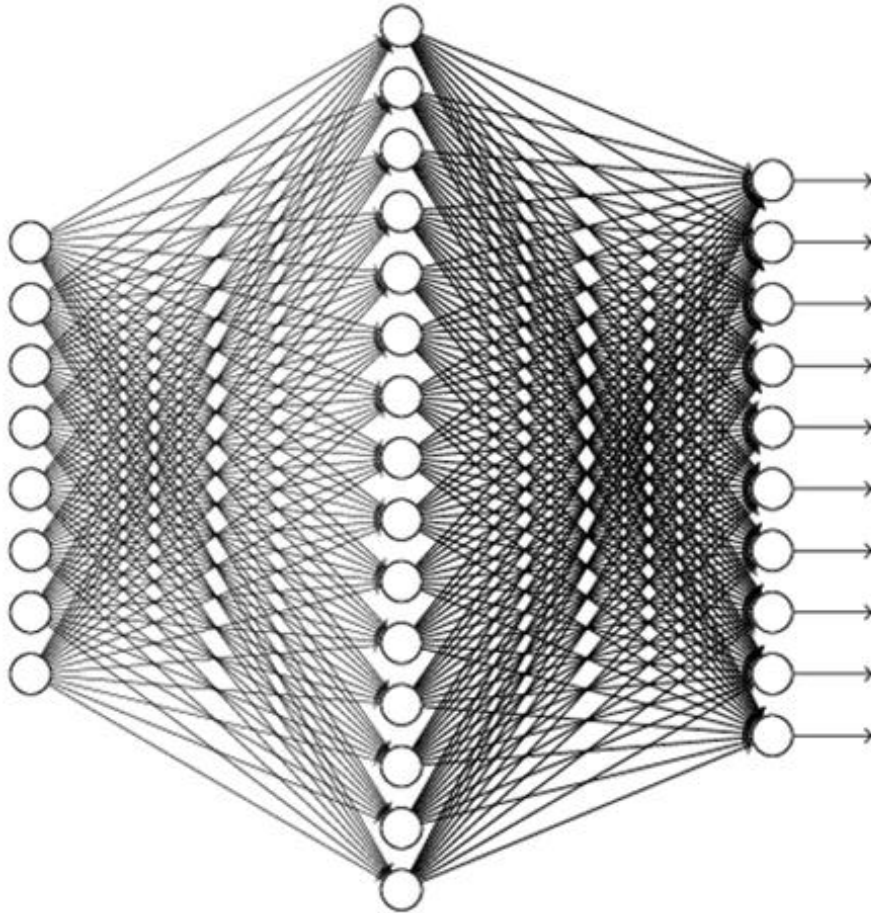
DEEP LEARNING FRAMEWORKS

- It's not practical to implement everything from scratch. Our numpy implementations were to know how NN works.
- There are many good deep learning frameworks.
- Deep learning is now in the phase of doing something with the frameworks and not from scratch to keep on going.
- Here are some of the leading deep learning frameworks:
 - Caffe/ Caffe2
 - CNTK
 - DL4j
 - Keras
 - Lasagne
 - mxnet
 - PaddlePaddle
 - TensorFlow
 - Theano
 - Torch/Pytorch

DEEP LEARNING FRAMEWORKS

- These frameworks are getting better month by month.
- How to choose deep learning framework:
 - Ease of programming (development and deployment)
 - Running speed
 - Truly open (open source with good governance)
- Programming frameworks can not only shorten your coding time but sometimes also perform optimizations that speed up your code.

SHALLOW VS DEEP NETWORK



Shallow Network

- A sufficiently wide neural network with just a single hidden layer can approximate any (reasonable) function given enough training data.
- **Very wide, shallow networks are very good at memorization, but not so good at generalization.**
- If you train the network with every possible input value, a super wide network could eventually memorize the corresponding output value that you want. But that's not useful because for any practical application you won't have every possible input value to train with.

SHALLOW VS DEEP NETWORK

Deep Network

The advantage of multiple layers is that they can learn features at various levels of abstraction.

Example: Image Classification

Layer 1: edges

Layer 2: collections of edges such as shapes

Layer 3: Eyes or noses

.....

Layer n: higher-order features -faces.

Multiple layers are much better at generalizing because they learn all the intermediate features between the raw data and the high-level classification.

MOTIVATION BEHIND CONVNETS

Consider an image of size $200 \times 200 \times 3$ (200 wide, 200 high, 3 color channels)

- a **single fully-connected neuron** in a first hidden layer of a regular Neural Network would have $200 \times 200 \times 3 = 120,000$ weights.

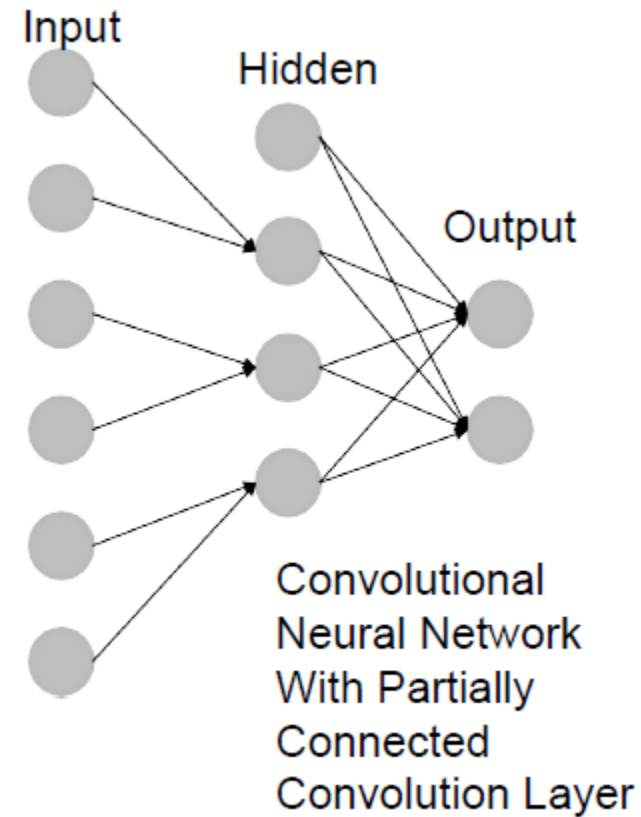
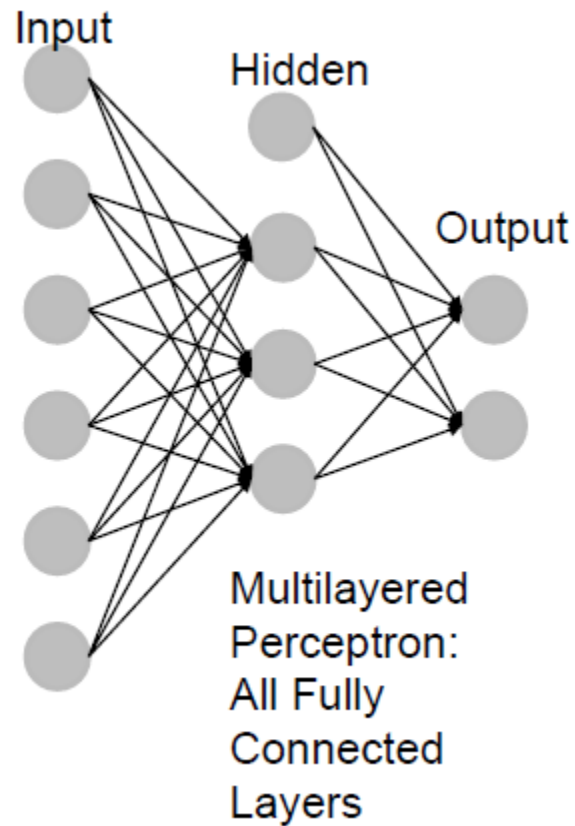
- Due to the presence of several such neurons, this full connectivity is waste and the huge number of parameters would quickly lead to overfitting

- However, in a ConvNet, the neurons in a layer will only be connected to a small region of the layer

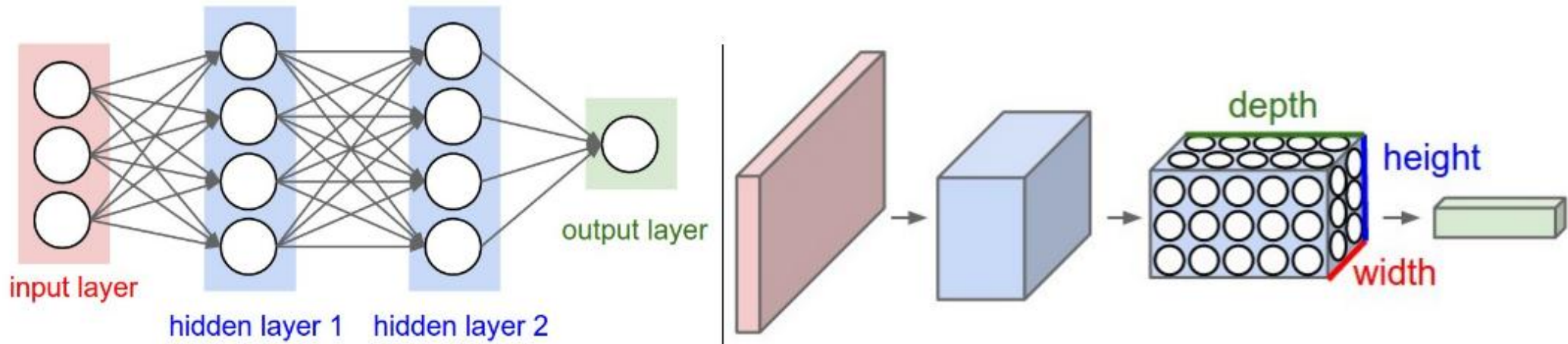
before it, instead of all of the neurons in a fully-connected manner.

- the final output layer would have dimensions $1 \times 1 \times N$, because by the end of the ConvNet architecture we will reduce the full image into a single vector of class scores (for N classes), arranged along the depth dimension

MLP VS CONVNET

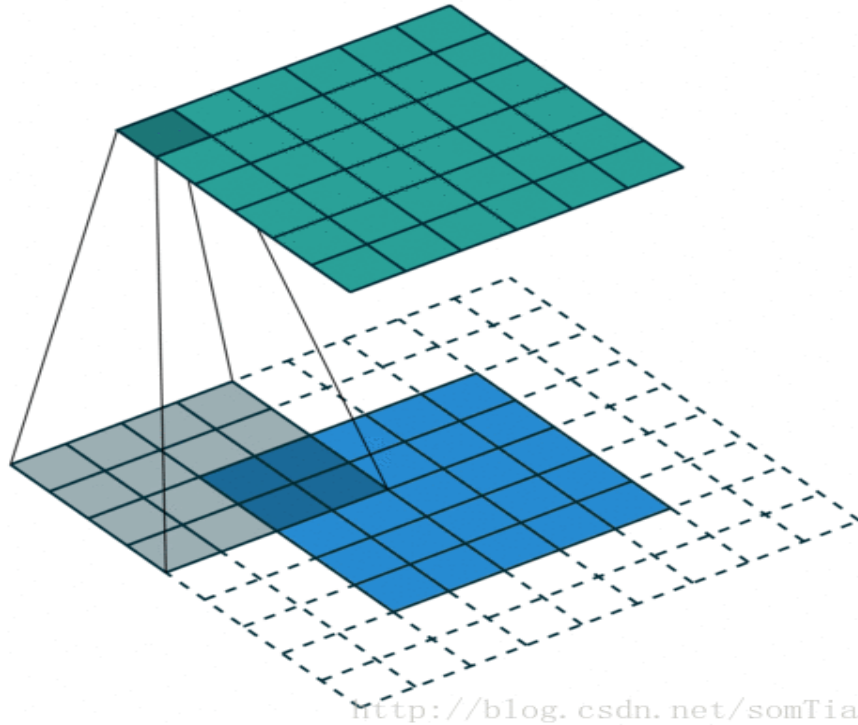


MLP VS CONVNET

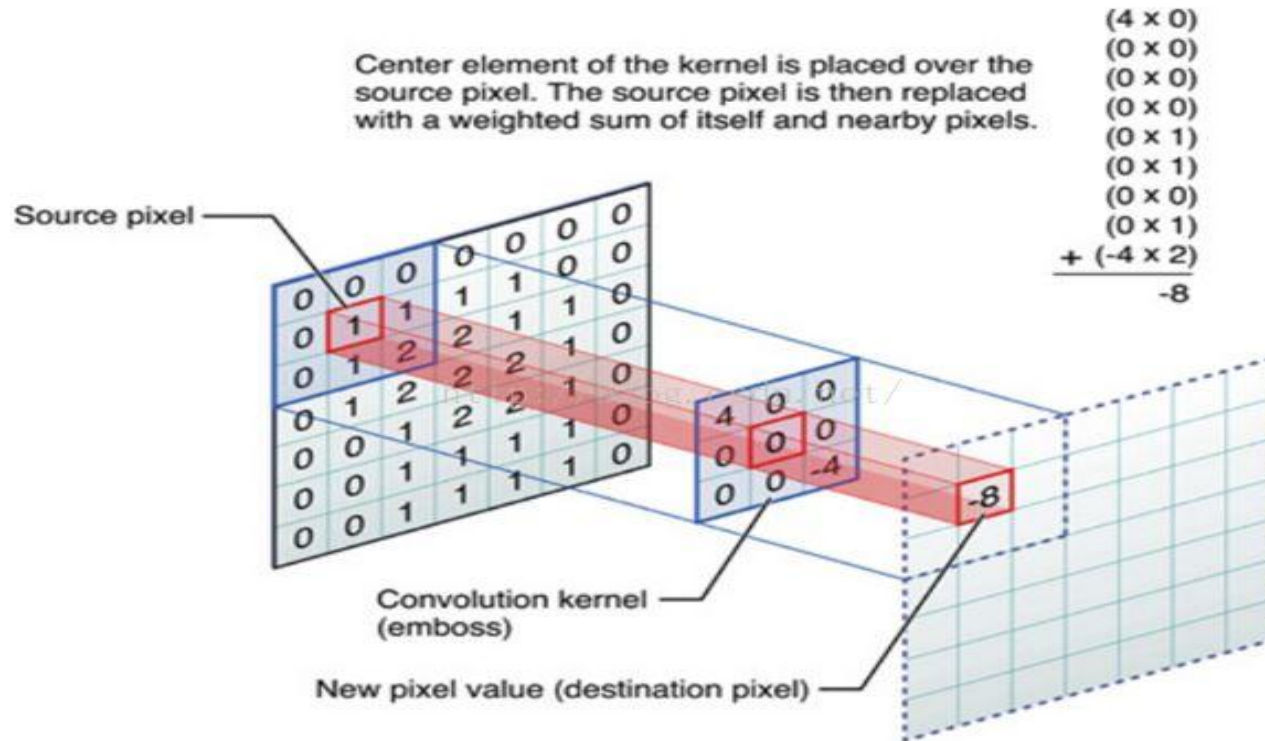


Left: A regular 3-layer Neural Network. Right: A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).

CONVOLUTIONAL KERNEL



CONVOLUTIONAL KERNEL



Padding on the input volume with zeros in such way that the conv layer does not alter the spatial dimensions of the input

CONVOLUTION

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern
(3 x 3).

CONVOLUTION

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Dot
product

3

-1

CONVOLUTION

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



CONVOLUTION

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

CONVOLUTION

stride=1

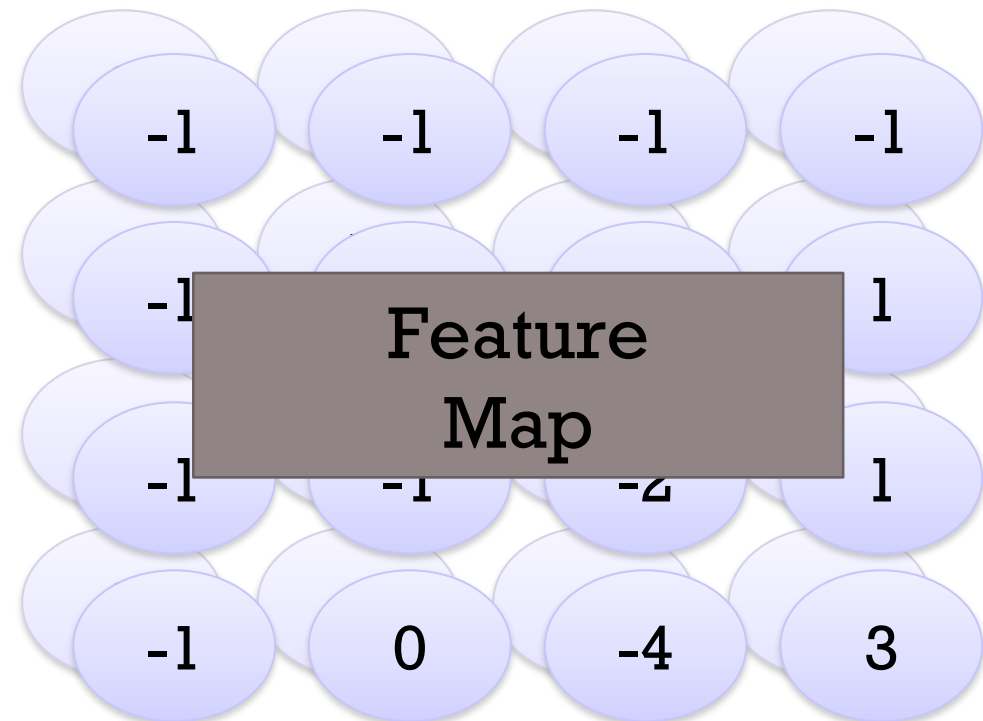
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

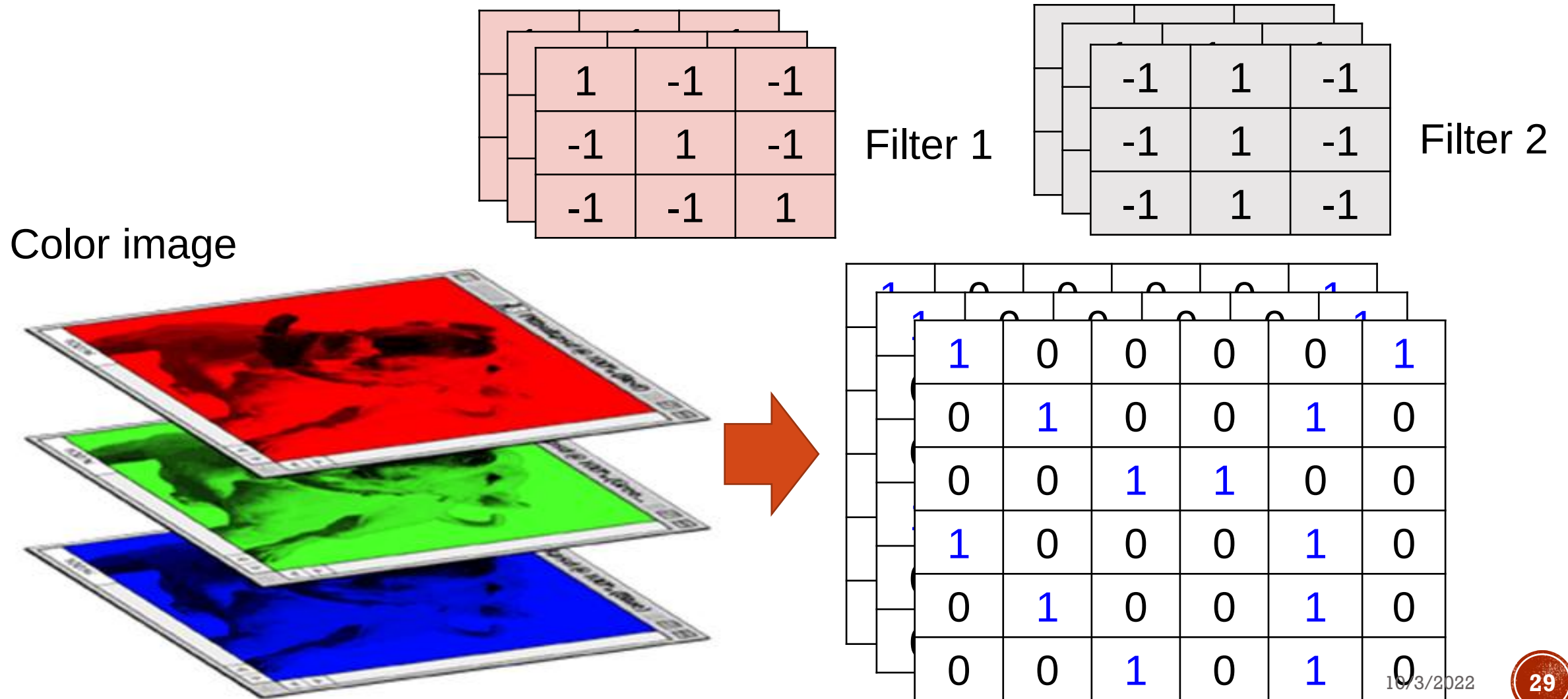
Repeat this for each filter



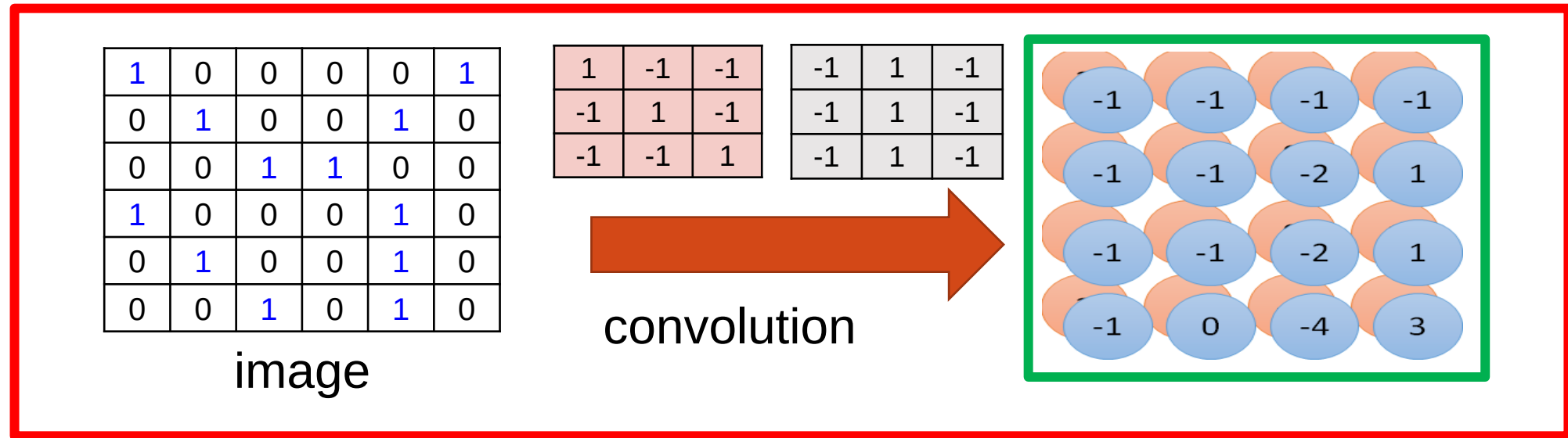
Two 4 x 4 images
Forming 2 x 4 x 4 matrix

10/3/2022

COLOR IMAGE: RGB 3 CHANNELS

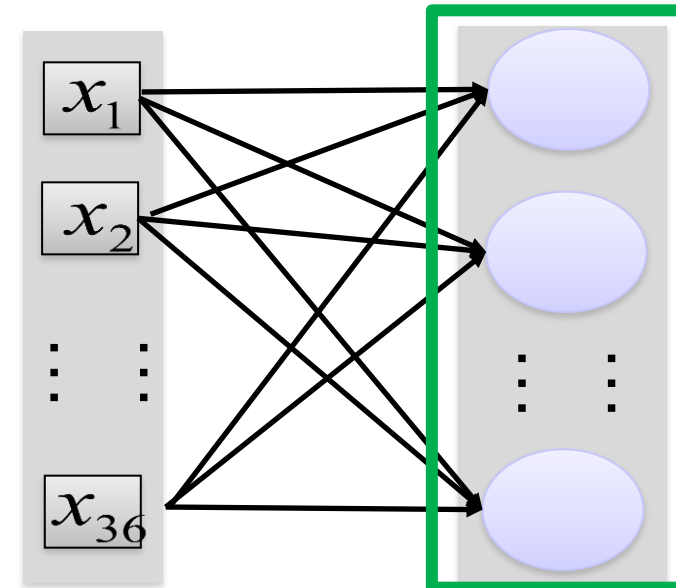


Convolution v.s. Fully Connected

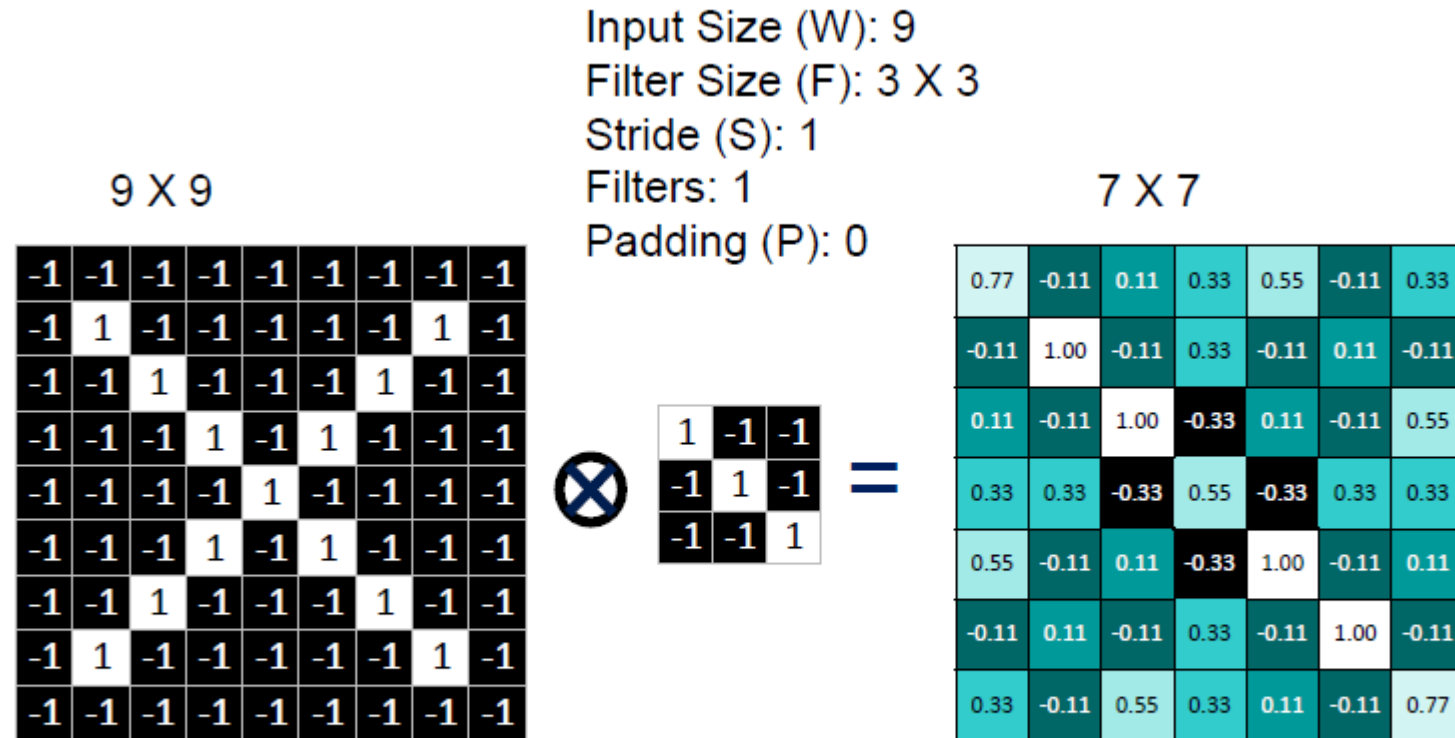


Fully-
connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



CONVOLUTIONAL LAYER – FILTERS – COMPUTATION EXAMPLE



$$\begin{aligned}\text{Feature Map Size} &= 1 + (W - F + 2P)/S \\ &= 1 + (9 - 3 + 2 \times 0)/1 = 7\end{aligned}$$

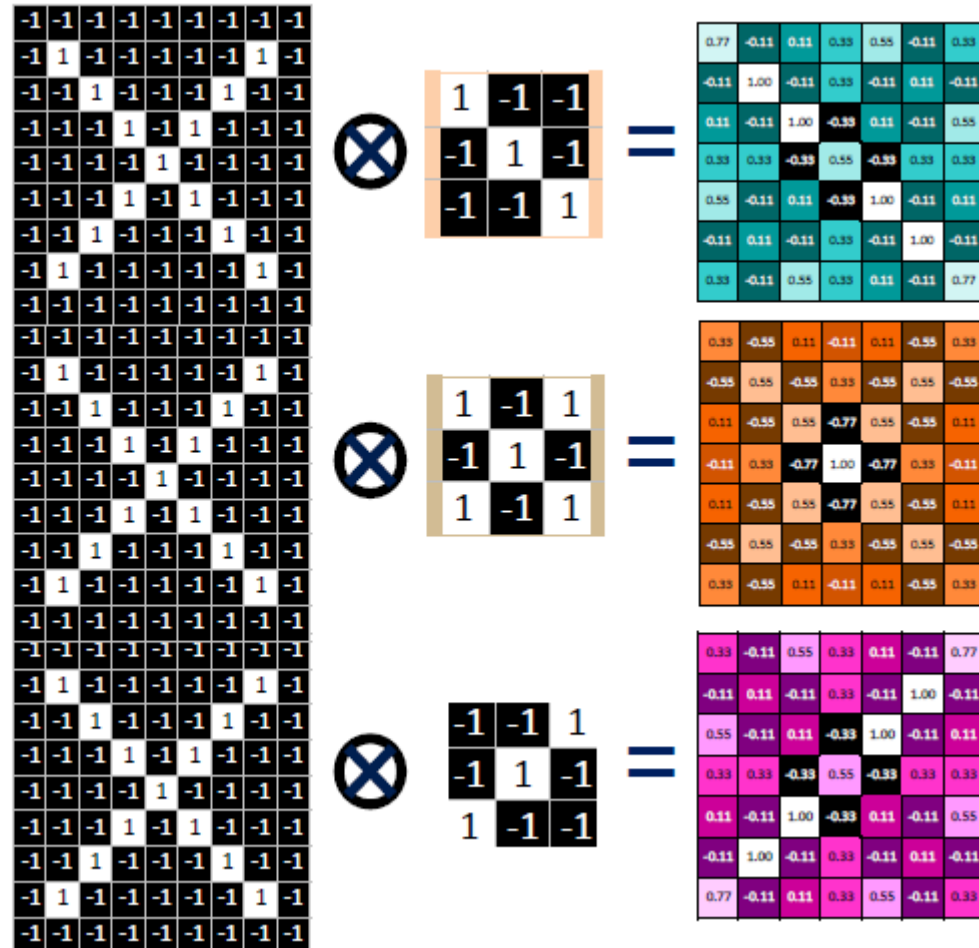
CONVOLUTIONAL LAYER – FILTERS – OUTPUT FEATURE MAP

- Output Feature Map of One complete convolution:

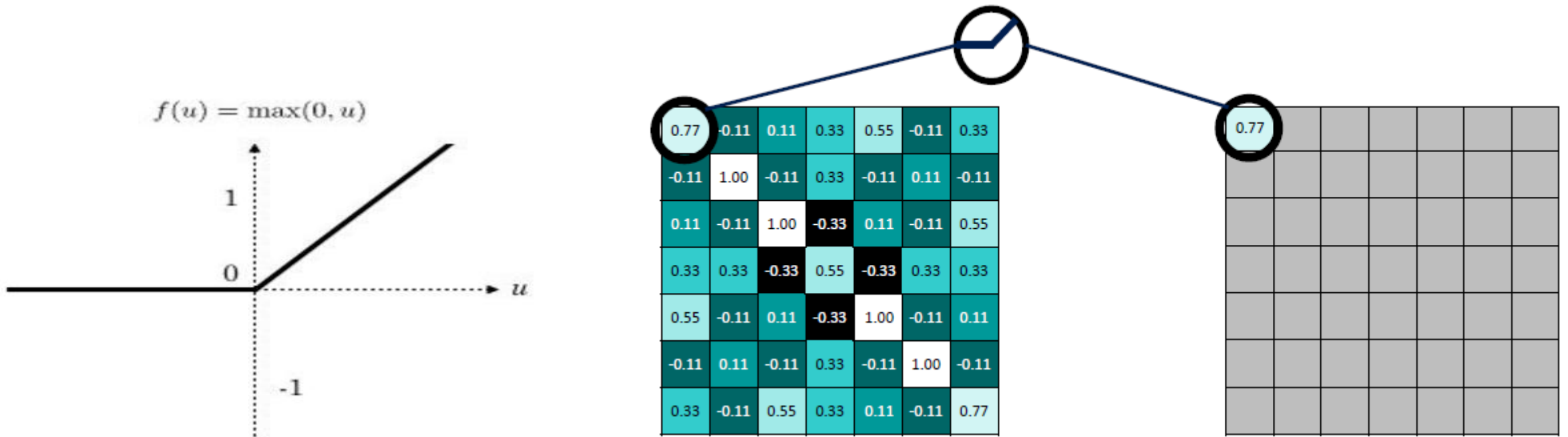
- Filters: 3
- Filter Size: 3 X 3
- Stride: 1

- Conclusion:

- Input Image:
9 X 9
- Output of Convolution:
7 X 7 X 3



RECTIFIED LINEAR UNIT (RELU)

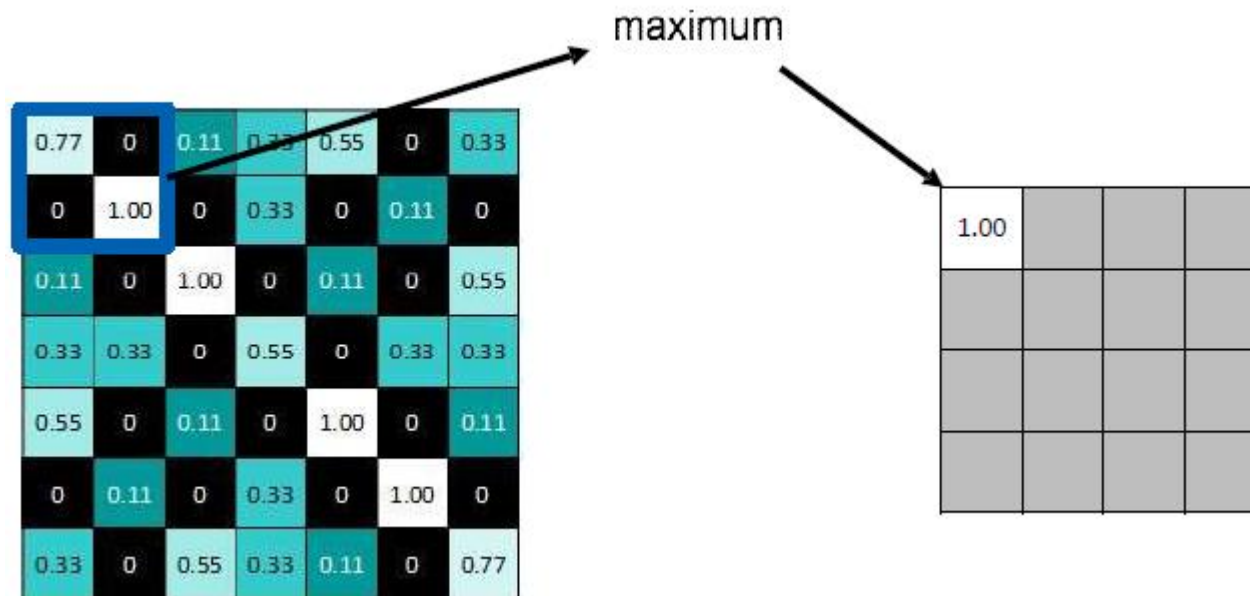


RECTIFIED LINEAR UNIT (RELU)

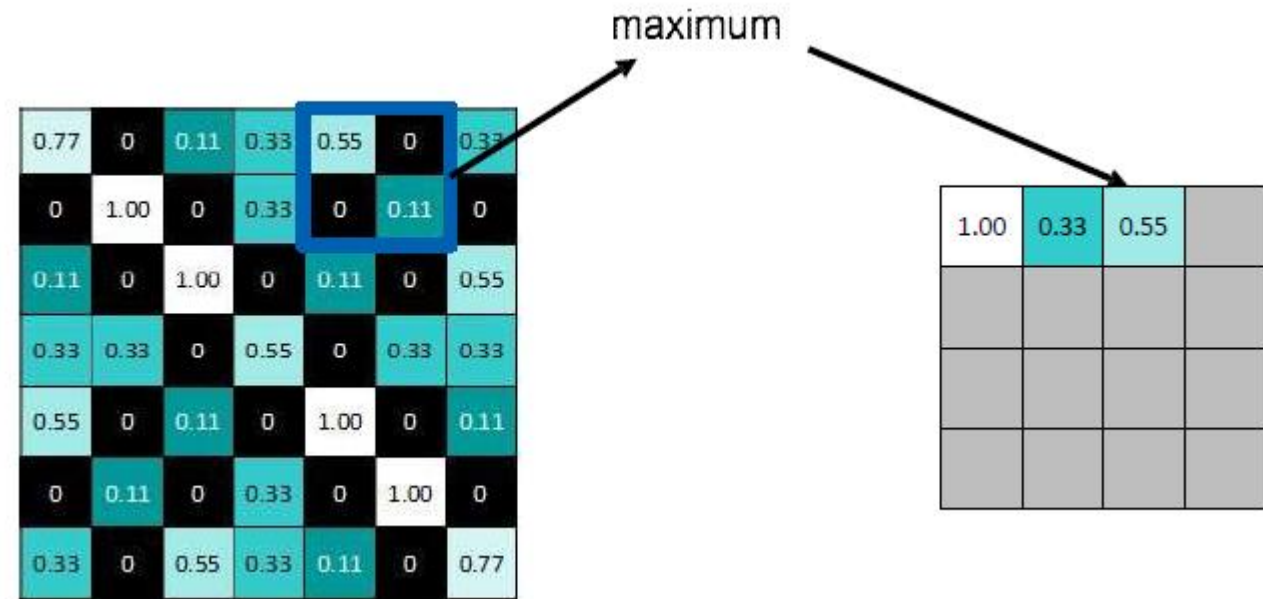
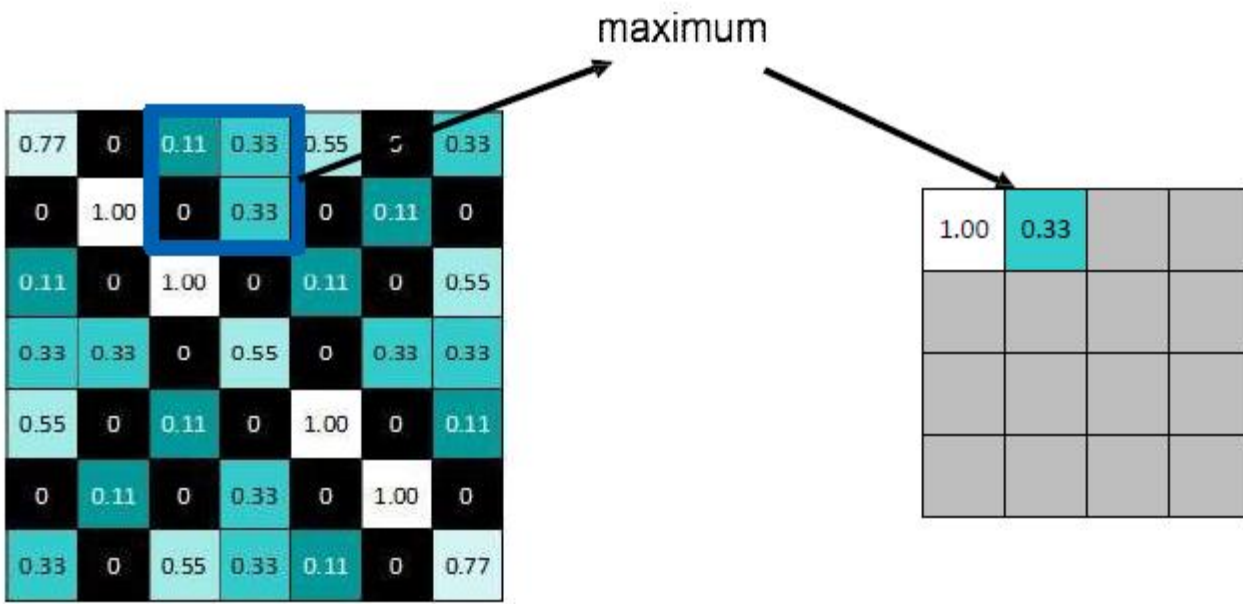


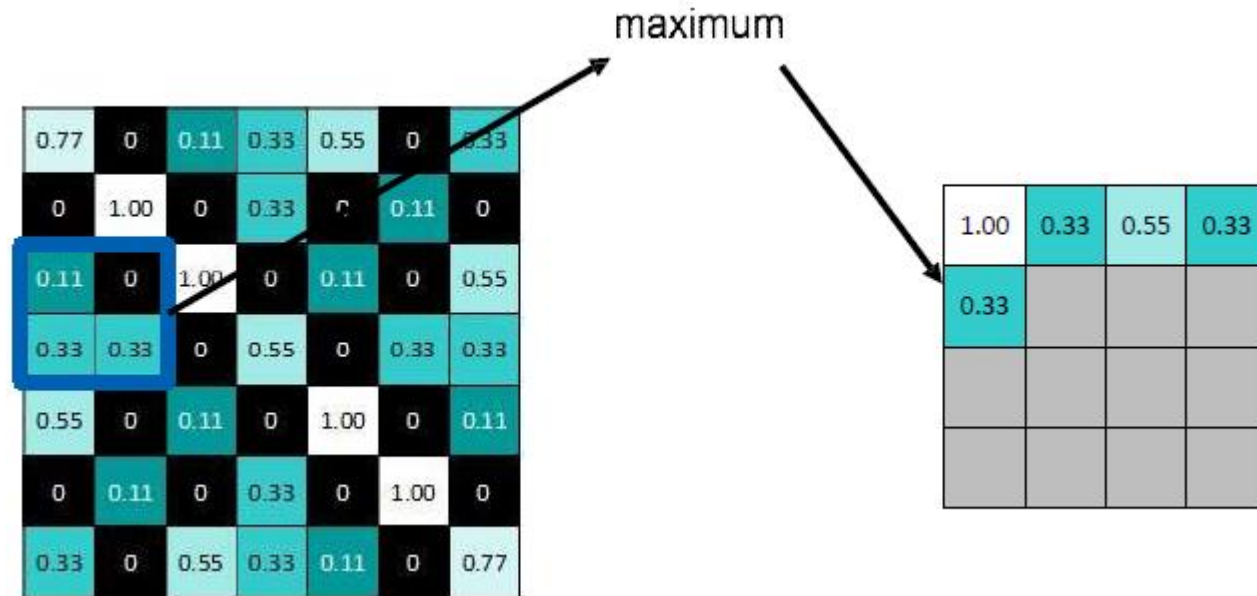
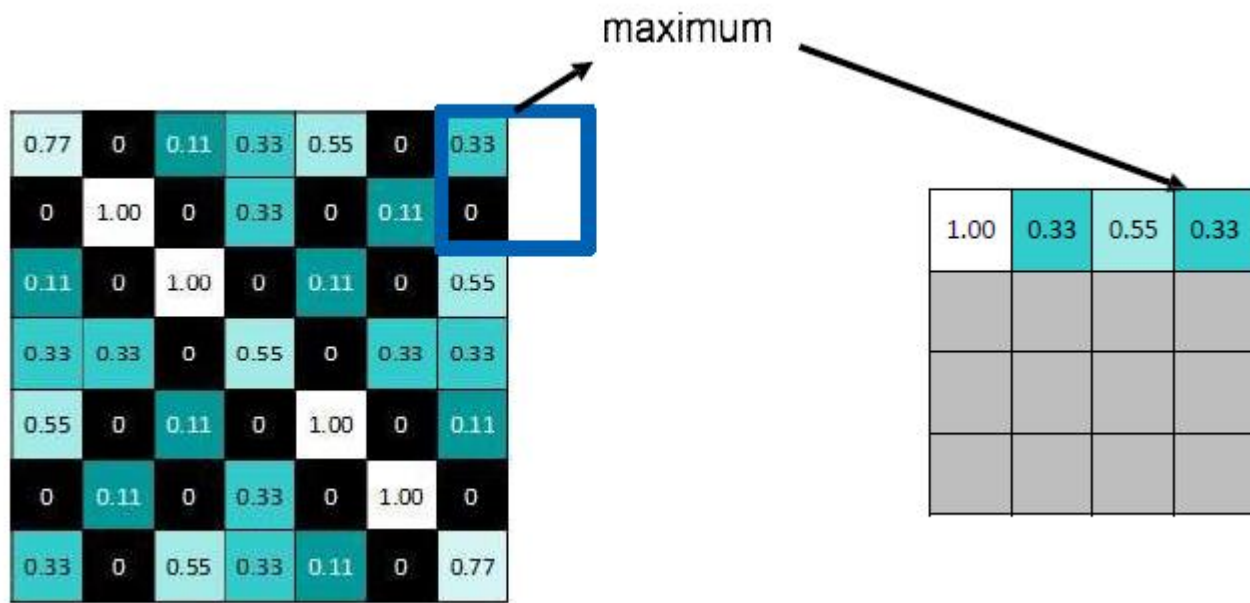
POOLING

- The pooling layers down-sample the previous layers feature map.
- Its function is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network
- The pooling layer often uses the Max operation to perform the downsampling process



Pooling Filter Size
= 2 X 2, Stride = 2





POOLING

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

max pooling

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

A stack of 3 larger images becomes a stack of smaller images.

0.77	0	0.11	0.33	0.55	0	0.11
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77



1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.33	0	0.11	0	0.11	0	0.55
0	0.55	0	0.33	0	0.55	0
0.11	0	0.55	0	0.55	0	0.11
0	0.11	0	1.00	0	0.11	0
0.33	0	0.55	0	0.55	0	0.33
0	0.33	0	0.33	0	0.33	0
0.33	0	0.11	0	0.11	0	0.33



0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

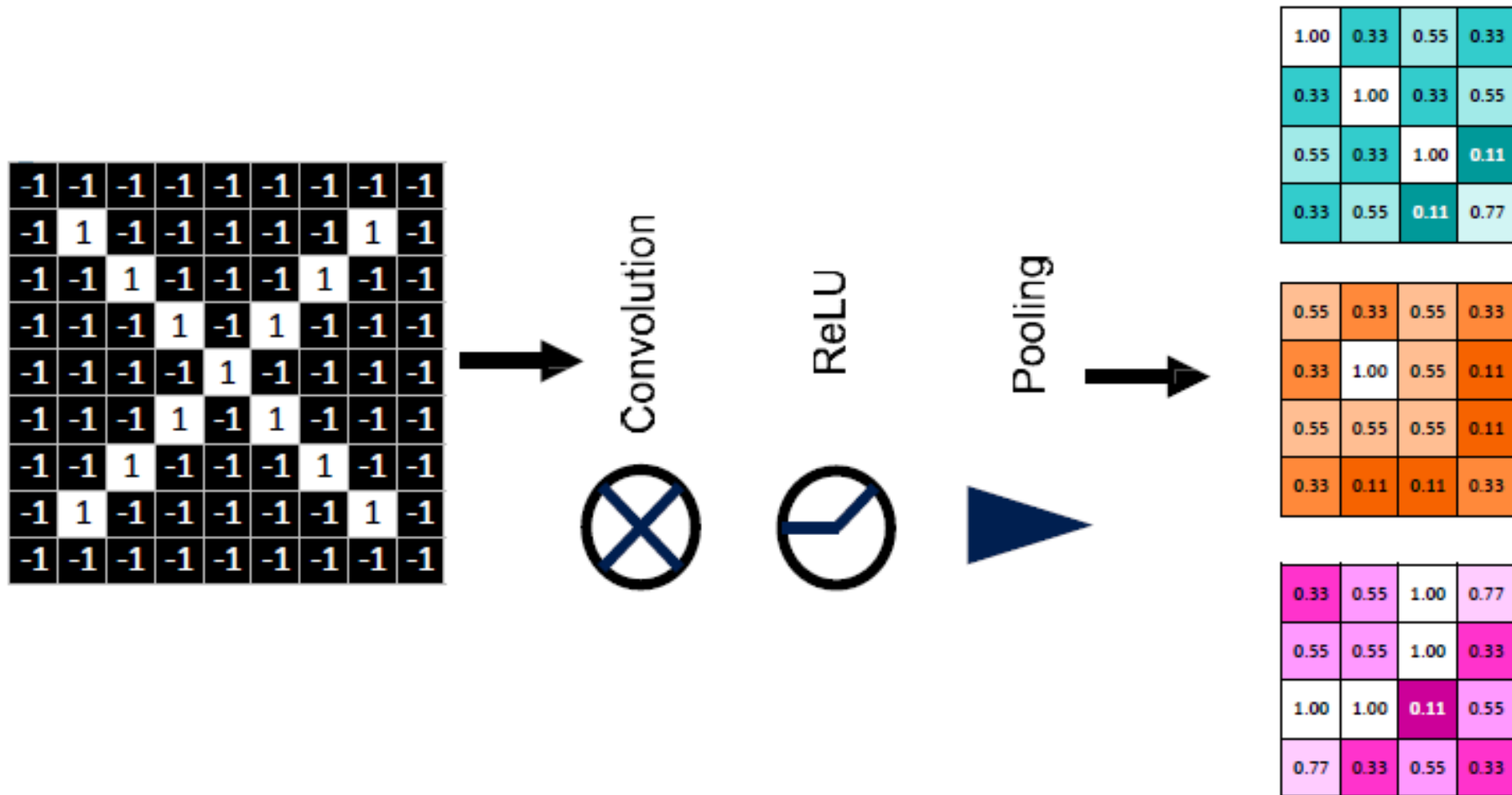
0.11	0	0.55	0.11	0.11	0	0.77
0	0.11	0	0.33	0	1.00	0
0.33	0	0.11	0	1.00	0	0.11
0.11	0.55	0	0.55	0	0.11	0.33
0.11	0	1.00	0	0.11	0	0.55
0	1.00	0	0.33	0	0.33	0
0.77	0	0.11	0.11	0.55	0	0.33



0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

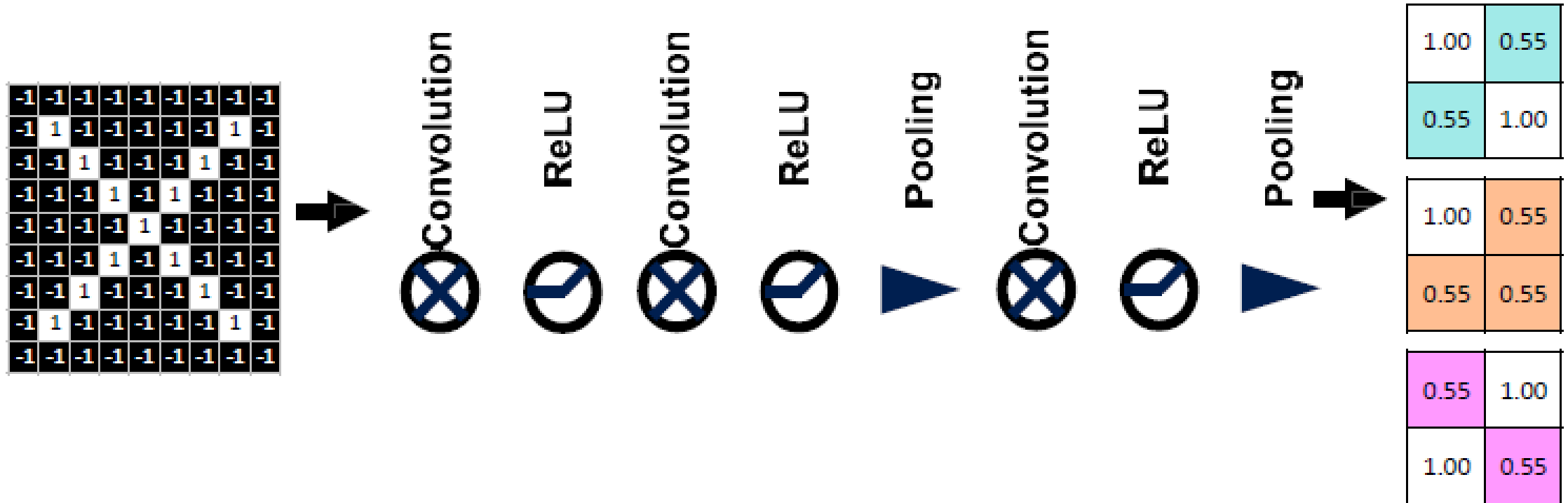
LAYERS GET STACKED

The output of one becomes the input of the next.



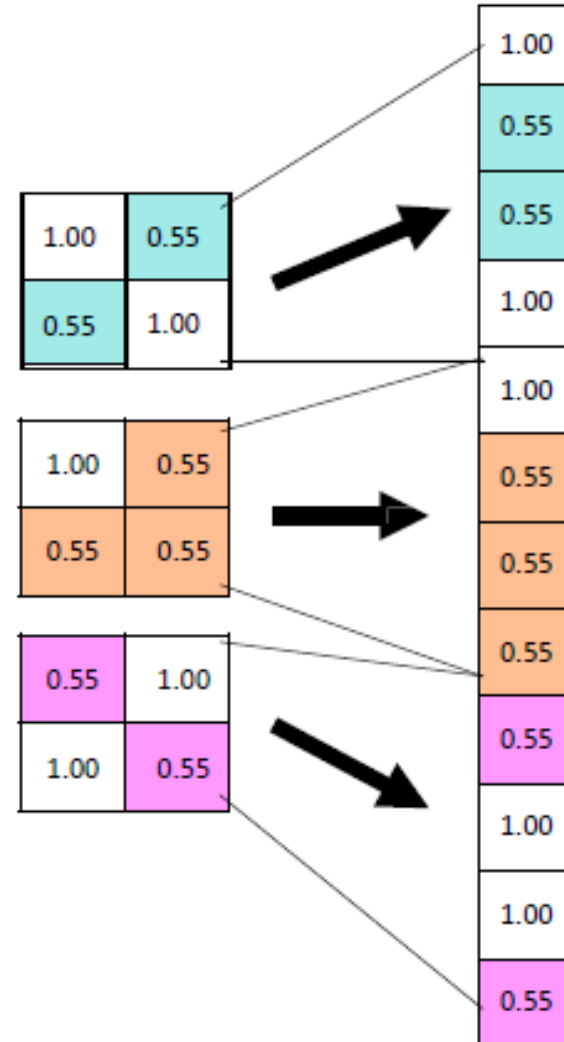
DEEP STAKING

Layers can be repeated several (or many) times.



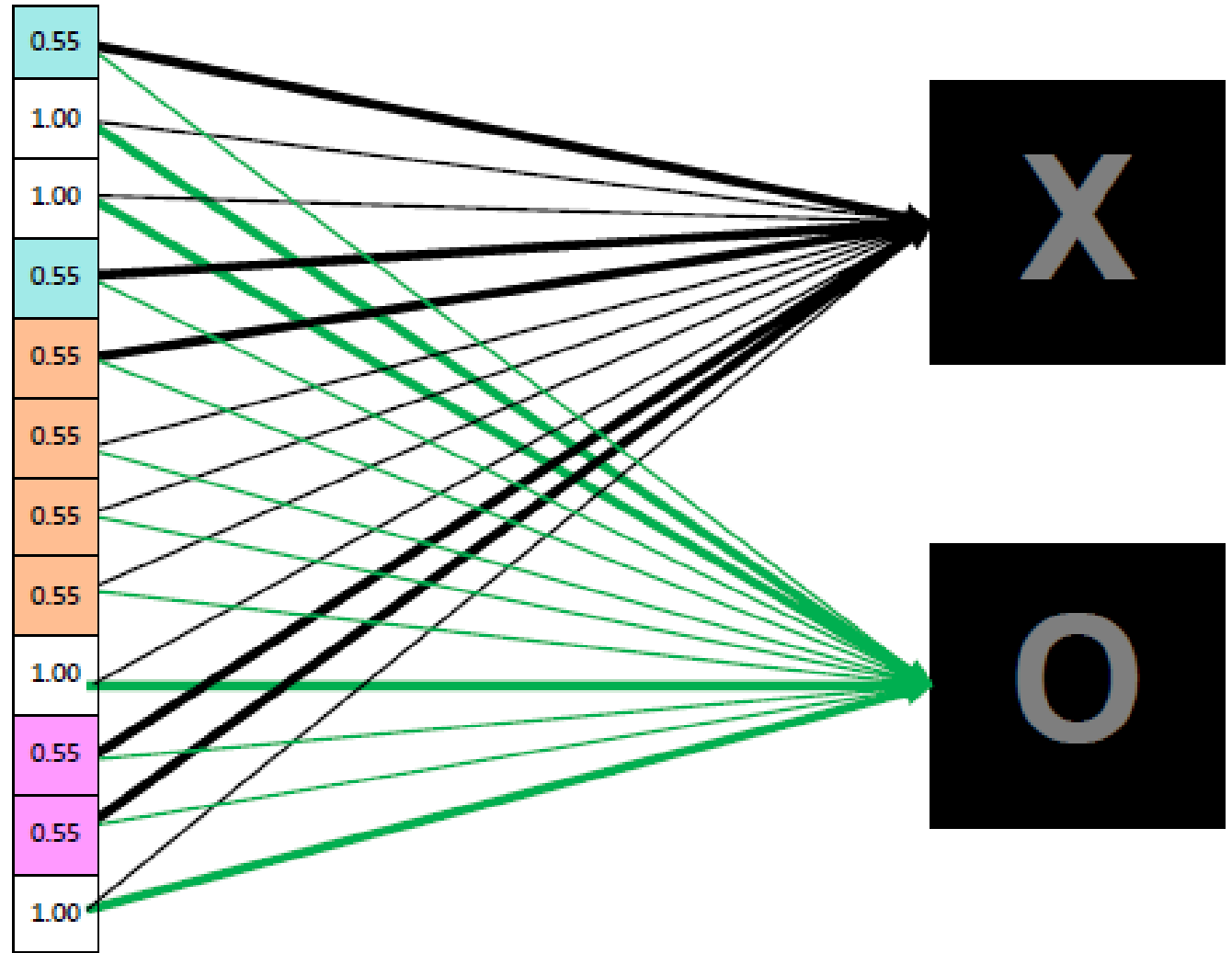
FULLY CONNECTED LAYER

- Fully connected layers are the normal flat feed-forward neural network layers.
- These layers may have a non-linear activation function or a softmax activation in order to predict classes.
- To compute our output, we simply rearrange the output matrices as a 1-D array.

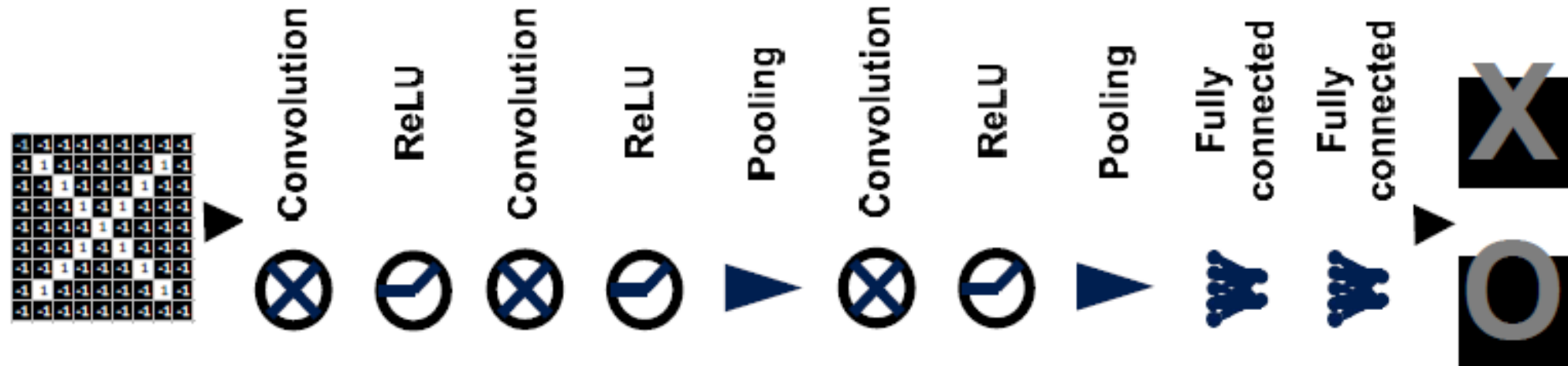


FC LAYER

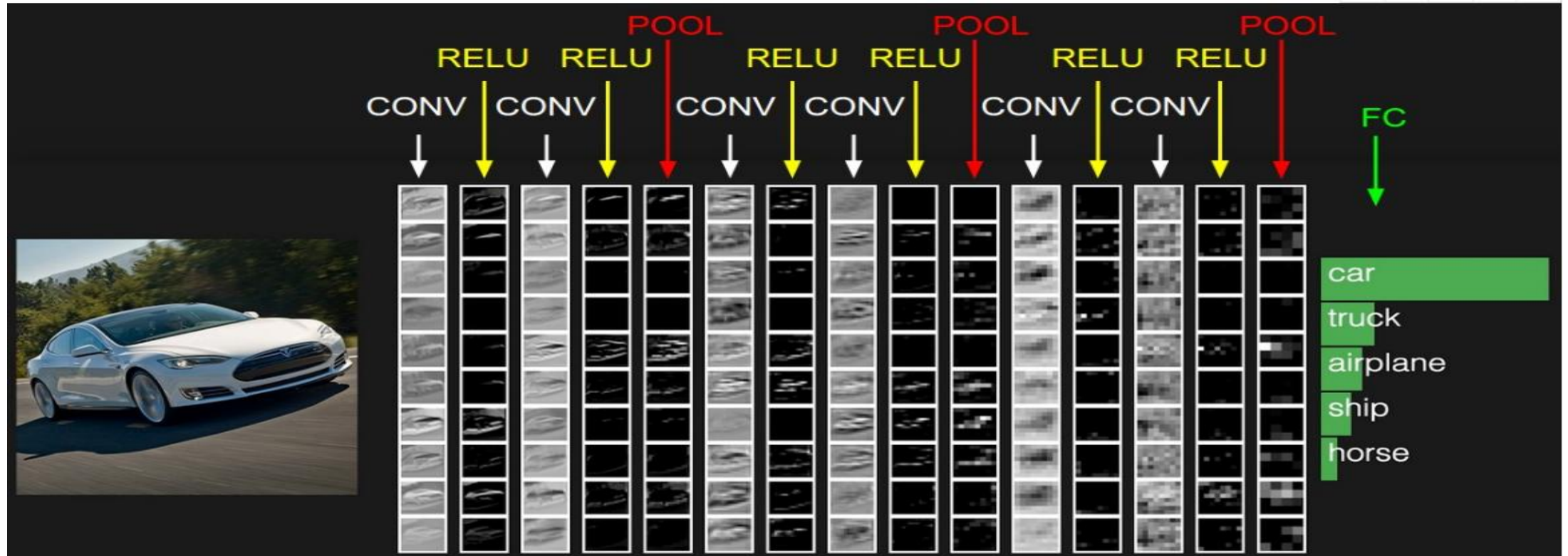
- A summation of product of inputs and weights at each output node determines the final prediction



PUTTING IT ALL TOGETHER

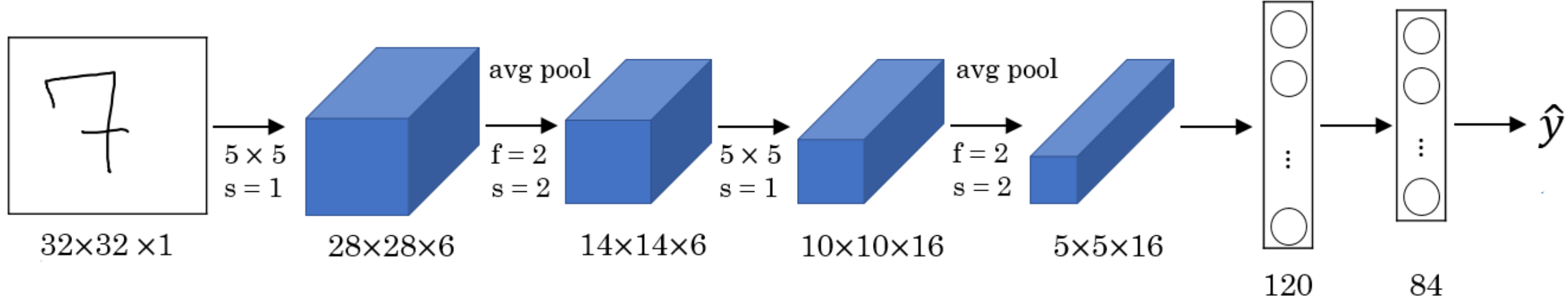


A SIMPLE CNN STRUCTURE



CONV: Convolutional kernel layer
RELU: Activation function
POOL: Dimension reduction layer
FC: Fully connection layer

LeNet



- The goal for this model was to identify handwritten digits in a 32x32x1 gray image.
- This model was published in 1998. The last layer wasn't using softmax back then.
- It has 60k parameters.
- The dimensions of the image decreases as the number of channels increases.
- Conv ==> Pool ==> Conv ==> Pool ==> FC ==> FC ==> softmax this type of arrangement is quite common.
- The activation function used in the paper was Sigmoid and Tanh. Modern implementation uses RELU in most of the cases.

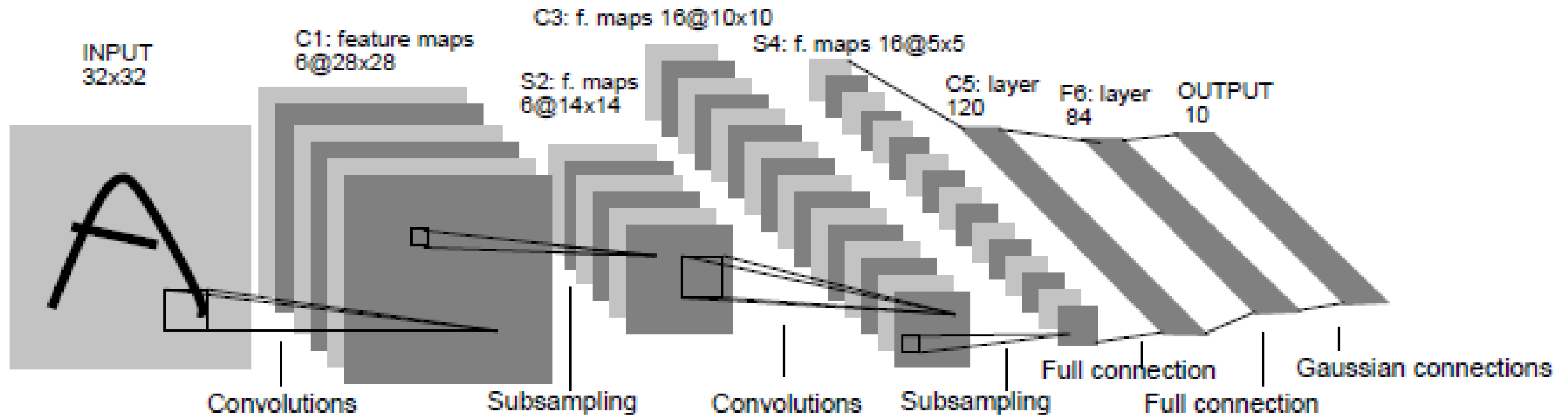


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

LeNet

Layer Type	Feature Map	Size	Kernel Size	Stride	Activation
Image	1	32×32	–	–	–
Convolution	6	28×28	5×5	1	tanh
Average Pooling	6	14×14	2×2	2	–
Convolution	16	10×10	5×5	1	tanh
Average Pooling	16	5×5	2×2	2	–
Fully Connected	–	120	–	–	tanh
Fully Connected	–	84	–	–	tanh
Fully Connected	–	10	–	–	softmax

LeNet

- “Gradient-Based Learning Applied To Document Recognition”.

UNIT-II

CONVOLUTION NEURAL NETWORK (CNN): Introduction to CNNs and their applications in computer vision, CNN basic architecture, Activation functions- sigmoid, tanh, ReLU, Leaky ReLU, Softmax layer, Types of pooling layers, Training of CNN in TensorFlow, various popular CNN architectures: VGG, Google Net, ResNet etc, Dropout, Normalization, Data augmentation.

Introduction to CNNs and Their Applications in Computer Vision

Convolutional Neural Networks (CNNs) are a specialized class of deep learning models designed primarily for processing data with a grid-like topology, such as images. They have revolutionized the field of computer vision by achieving remarkable accuracy in tasks like image classification, object detection, segmentation, and facial recognition. CNNs are inspired by the structure of the visual cortex in animals, where neurons respond to overlapping regions of the visual field. This biological motivation helps CNNs recognize patterns and features in images effectively.

The key idea behind CNNs is the convolution operation, which replaces traditional matrix multiplication in at least one of the layers of a neural network. This operation uses small filters or kernels to scan through the input image and extract local features such as edges, textures, or colors. These features are then passed through layers of increasing depth, where more abstract patterns are recognized, such as shapes or entire objects.

One of the main advantages of CNNs is their parameter efficiency. Unlike fully connected networks that treat all pixels equally and require a large number of weights, CNNs share weights through filters, significantly reducing the number of parameters. This makes CNNs more computationally efficient and less prone to overfitting, especially on high-dimensional input like images.

CNNs are structured hierarchically, with early layers capturing low-level features (edges, corners), and deeper layers recognizing high-level semantic features (like eyes, wheels, or digits). This layered feature learning process allows CNNs to automatically extract features without the need for manual feature engineering.

In terms of applications, CNNs have become the backbone of modern computer vision systems. In image classification, CNNs can distinguish between thousands of categories such as cats, dogs, and airplanes. In object detection, they not only classify objects but also localize them using bounding boxes. Semantic segmentation uses CNNs to label each pixel in an image with a class label. In medical imaging, CNNs are used to identify tumors in X-rays, MRIs, or CT scans. They are also used in autonomous vehicles to detect pedestrians, lane markings, and traffic signs.

Further applications include facial recognition, optical character recognition (OCR), gesture recognition, and style transfer. CNNs also play a critical role in content-based image retrieval systems and are increasingly being used in video analysis, such as action recognition and video classification.

CNNs provide a powerful framework for handling image and video data by leveraging spatial hierarchies and local feature extraction. Their success in real-world applications continues to grow, making them a fundamental component of modern artificial intelligence systems.

CNN basic architecture

The basic architecture of a Convolutional Neural Network (CNN) is composed of a sequence of layers, each of which transforms the input data into increasingly abstract and informative representations. The architecture typically includes three main types of layers: convolutional layers, pooling layers, and fully connected layers. Each of these plays a distinct role in processing the input data, particularly images.

The input layer of a CNN receives raw pixel values from an image. For example, a grayscale image of size 28×28 is represented as a matrix of pixel intensity values, while a color image has three channels (Red, Green, Blue), forming a 3D tensor.

The convolutional layer is the core building block of a CNN. It uses small learnable filters (e.g., 3×3 or 5×5) that are convolved over the input to produce feature maps. Each filter is trained to detect a specific feature such as vertical edges, horizontal edges, or patterns. The convolution operation preserves the spatial relationship between pixels by learning local patterns, which makes CNNs particularly powerful in visual tasks.

After the convolution operation, the output is passed through a non-linear activation function like ReLU (Rectified Linear Unit), which introduces non-linearity into the model, allowing it to learn complex mappings.

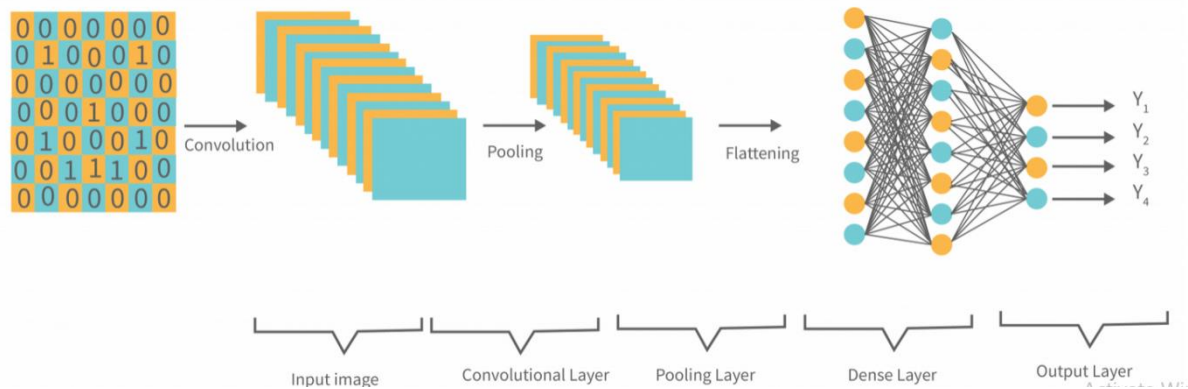
Next comes the pooling layer, which performs downsampling. Pooling reduces the spatial dimensions (width and height) of the feature maps while retaining the most important information. This not only reduces computational cost but also provides translation invariance. Common pooling techniques include max pooling and average pooling.

The flattening layer converts the 2D feature maps into a 1D vector, which is then fed into one or more fully connected (dense) layers. These layers are similar to those in traditional neural networks and are used to combine the features learned by the convolutional and pooling layers to make final predictions.

Finally, the output layer produces the prediction results. For classification tasks, it often uses a softmax activation function, which converts the output into a probability distribution across different classes.

A CNN architecture may also include regularization techniques such as dropout to prevent overfitting and batch normalization to stabilize training. The design of the architecture (number of layers, filter sizes, stride, padding, etc.) is usually tailored to the specific problem and dataset.

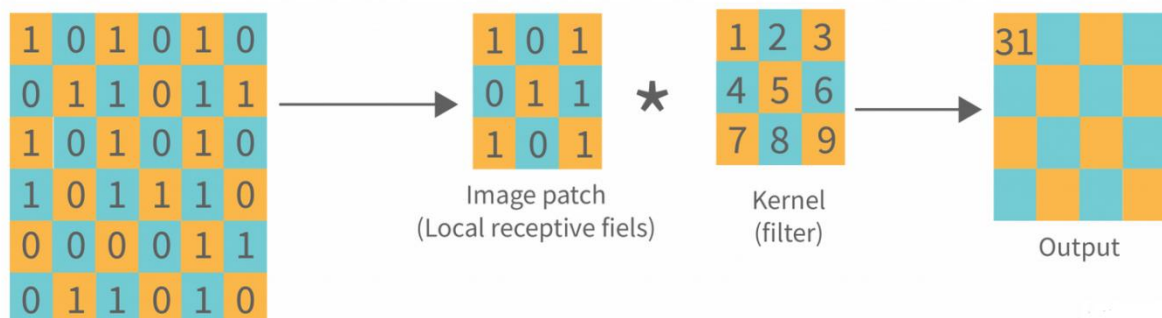
To summarize, the basic architecture of a CNN transforms raw image input through a series of convolution, activation, and pooling layers into a meaningful output. This architecture allows the model to learn hierarchical representations of data — from low-level features like edges to high-level semantics like objects — making CNNs the foundation of modern computer vision solutions.



The ConvNet's job is to compress the images into a format that is easier to process while preserving elements that are important for obtaining a decent prediction. This is critical for designing an architecture that is capable of learning features while also being scalable to large datasets.

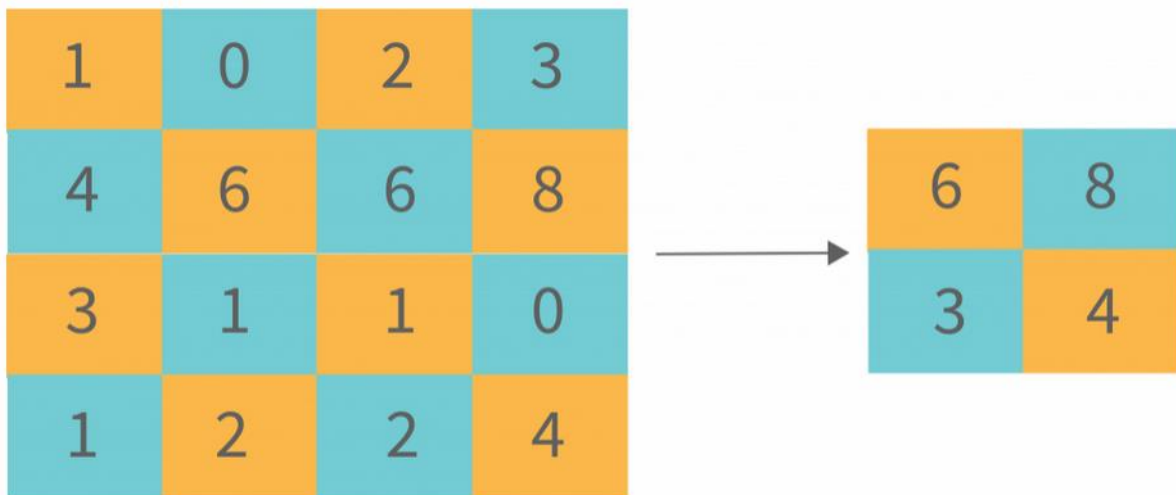
A convolutional neural network, ConvNets in short has three layers which are its building blocks, let's have a look:

Convolutional Layer (CONV): They are the foundation of CNN, and they are in charge of executing convolution operations. The Kernel/Filter is the component in this layer that performs the convolution operation (matrix). Until the complete image is scanned, the kernel makes horizontal and vertical adjustments dependent on the stride rate. The kernel is less in size than a picture, but it has more depth. This means that if the image has three (RGB) channels, the kernel height and width will be modest spatially, but the depth will span all three.

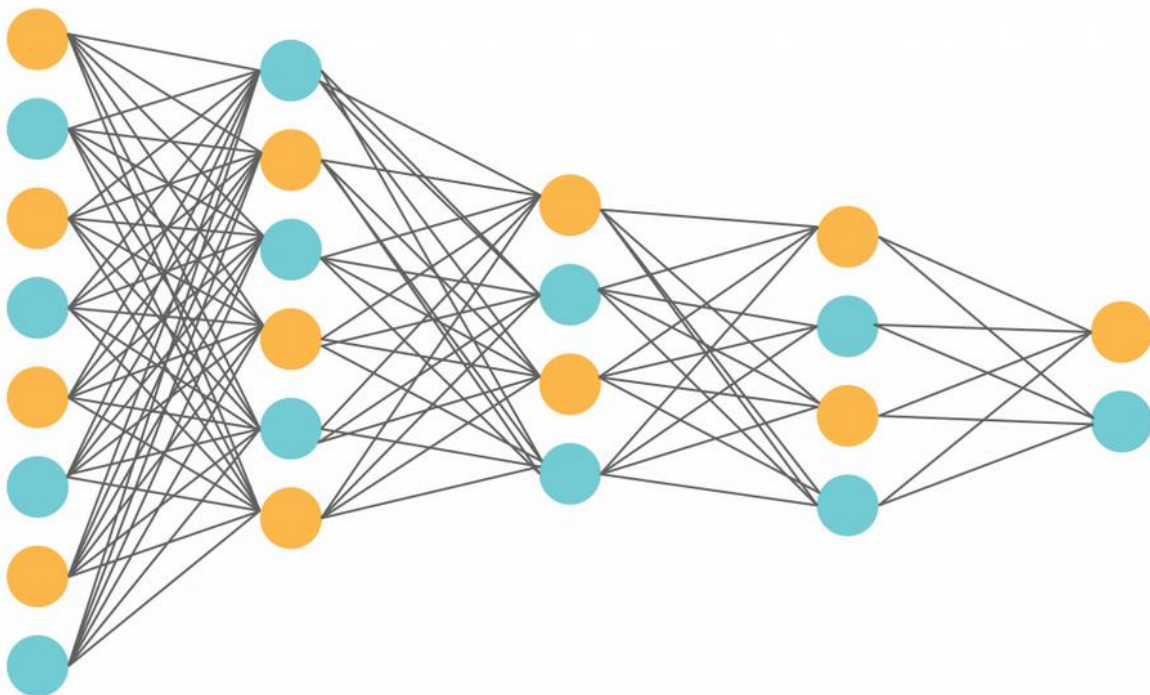


Other than convolution, there is another important part of convolutional layers, known as the Non-linear activation function. The outputs of the linear operations like convolution are passed through a non-linear activation function. Although smooth nonlinear functions such as the sigmoid or hyperbolic tangent (tanh) function were formerly utilized because they are mathematical representations of biological neuron actions. The rectified linear unit (ReLU) is now the most commonly used non-linear activation function. $f(x) = \max(0, x)$

Pooling Layer (POOL): This layer is in charge of reducing dimensionality. It aids in reducing the amount of computing power required to process the data. Pooling can be divided into two types: maximum pooling and average pooling. The maximum value from the area covered by the kernel on the image is returned by max pooling. The average of all the values in the part of the image covered by the kernel is returned by average pooling.



Fully Connected Layer (FC): The fully connected layer (FC) works with a flattened input, which means that each input is coupled to every neuron. After that, the flattened vector is sent via a few additional FC layers, where the mathematical functional operations are normally performed. The classification procedure gets started at this point. FC layers are frequently found near the end of CNN architectures if they are present.



Along with the above layers, there are some additional terms that are part of a CNN architecture.

Activation Function: The last fully connected layer's activation function is frequently distinct from the others. Each activity necessitates the selection of an appropriate activation function. The softmax function, which normalizes output real values from the last fully connected layer to target class probabilities, where each value ranges between 0 and 1 and all values total to 1, is an activation function used in the multiclass classification problem.

Dropout Layers: The Dropout layer is a mask that nullifies some neurons' contributions to the following layer while leaving all others unchanged. A Dropout layer can be applied to the input

vector, nullifying some of its properties; however, it can also be applied to a hidden layer, nullifying some hidden neurons. Dropout layers are critical in CNN training because they prevent the training data from overfitting. If they aren't there, the first batch of training data has a disproportionately large impact on learning. As a result, learning of traits that occur only in later samples or batches would be prevented.

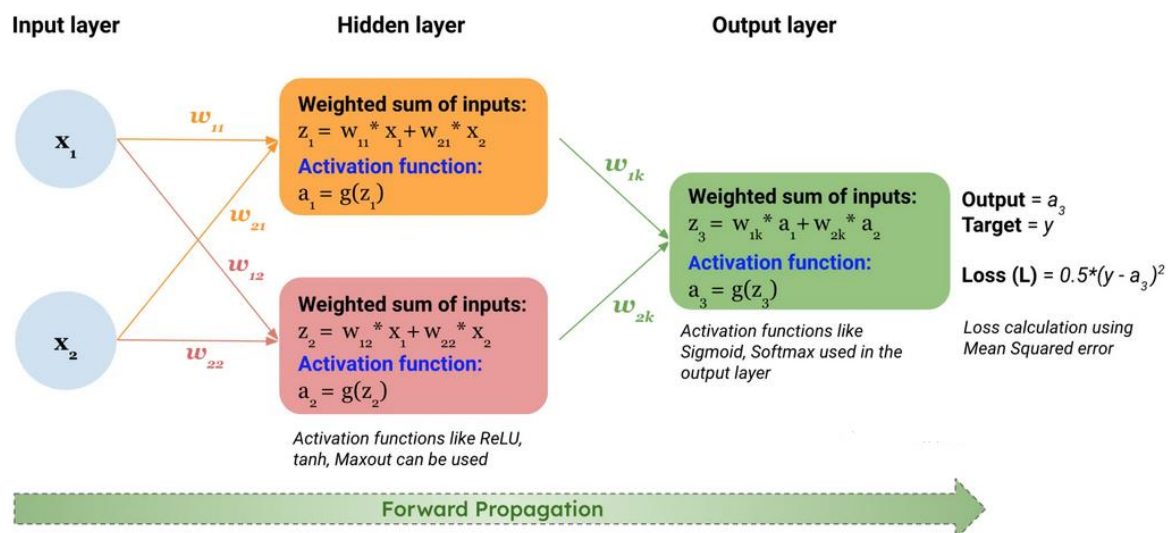
Advantages of CNN Architecture

Following are some of the advantages of a Convolutional Neural Network:

- CNN is computationally efficient.
- It performs parameter sharing and uses special convolution and pooling algorithms. CNN models may now run on any device, making them globally appealing.
- It finds the relevant features without the need for human intervention.
- It can be utilized in a variety of industries to execute key tasks such as facial recognition, document analysis, climate comprehension, image recognition, and item identification, among others.
- By feeding your data on each level and tuning the CNN a little for a specific purpose, you can extract valuable features from an already trained CNN with its taught weights.
-

Activation functions- sigmoid, tanh, ReLU, Leaky ReLU, Softmax layer

Activation functions are a critical component of neural networks, including CNNs. They determine whether a neuron should be activated or not by introducing non-linearity into the model. Without activation functions, the network would behave like a linear regression model, unable to capture complex patterns.



Activation functions serve two main purposes:

1. **Introduce Non-linearity:** Without non-linearity, a neural network would behave like a linear model, no matter how deep it is. Activation functions allow the network to learn and represent complex, non-linear mappings between inputs and outputs.
2. **Control Neuron Activation:** Activation functions control the firing behavior of neurons. Depending on the activation function's output, a neuron might become activated (output a non-zero value) or remain inactive (output zero).

Typically, same activation function is applied to all the hidden layers, while the output layer uses a different activation function, based on the type of prediction model aims to make.

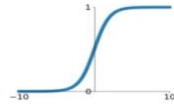
Different types of Activation Functions:

Activation functions, which are popularly used in neural network models, are shown in the figure below.

Activation Functions

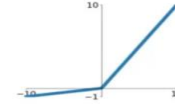
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



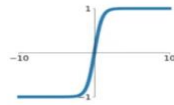
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

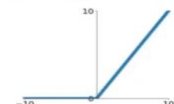


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

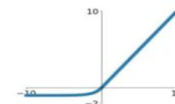
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



The sigmoid function is defined as $\sigma(x) = \frac{1}{1+e^{-x}}$.

It maps input values into the range (0, 1), making it useful for binary classification problems. However, it suffers from the vanishing gradient problem, where gradients become too small for large or small input values, slowing down or stopping learning.

The tanh (hyperbolic tangent) function is given by $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$.

It maps values to the range (-1, 1), and like sigmoid, it has a smooth gradient, but with a zero-centered output, which helps in optimization. Still, it suffers from vanishing gradients for large input values.

The ReLU (Rectified Linear Unit) function is defined as $f(x) = \max(0, x)$.

It has become the most commonly used activation in CNNs due to its simplicity and effectiveness. It allows the network to converge faster and avoids the vanishing gradient issue. However, ReLU can suffer from the “dying ReLU” problem, where neurons get stuck during training and output zero for all inputs.

To address this, Leaky ReLU introduces a small slope for negative input values, defined as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

where α is a small constant like 0.01. This modification allows negative input values to propagate and prevents neurons from dying.

For classification tasks, especially multi-class, the Softmax function is used in the final layer. It converts a vector of raw scores into probabilities by emphasizing the highest values and suppressing

the others. Each output is in the range (0,1) and sums to 1 across all classes, making it ideal for deciding the most likely class.

Pros and Cons of different Activation Functions

Used in layer	Activation Function	Details	Pros	Cons
Hidden / Output	Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$ Output range: [0,1]	- Smooth activation that outputs values between 0 and 1, making it suitable for binary classification - Historically popular	- Vanishing gradient problem due to saturated neurons - Output not zero-centered as sigmoid outputs are always positive - exp() operation is computationally intensive
Hidden	Tanh (Hyperbolic tangent)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Output range: [-1,1]	- Zero centered outputs that help networks train faster	- Suffers with vanishing gradient problem when saturated
Hidden	ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$ Output range: [0, ∞)	- Does not saturate: avoids vanishing gradient issues for positive inputs - Computationally efficient since only certain number of neurons are activated at the same time - Much faster convergence compared to sigmoid/tanh	- Output not zero-centered - Prone to a "dying ReLU" problem, where neurons can get stuck during training and never activate again, leading to a dead neuron that doesn't update its weights.

Used in layer	Activation Function	Details	Pros	Cons
Hidden	Leaky ReLU	$f(x) = \max(0.01x, x)$ Output range: $(-\infty, \infty)$	- All benefits of ReLU - Addresses the "dying ReLU" problem by allowing a small gradient for negative inputs. Helps with training deeper networks	- Not as standardized as ReLU, and the slope of the leaky part is typically a hyperparameter (α) that needs tuning. This is also referred to as Parametric ReLU $f(x) = \max(\alpha x, x)$
Hidden	ELU (Exponential Linear Unit)	where $\alpha > 0$: $f(x) = \begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$ α is commonly chosen as 1 Output range: $(-\alpha, \infty)$	- All benefits of ReLU - Zero centered outputs that help networks train faster - ELUs saturate to a negative value when the argument gets smaller becoming more robust to noise	- Computationally more expensive due to exponential operation

Activation functions are essential for learning complex, non-linear relationships in data. Choosing the right activation function affects the learning efficiency, convergence speed, and overall performance of CNN models.

Types of pooling layers

Pooling layers are a fundamental part of Convolutional Neural Networks (CNNs). Their main function is to progressively reduce the spatial dimensions (width and height) of the feature maps, which helps in reducing the number of parameters and computations in the network. Pooling also provides translation invariance, meaning that small shifts in the input image do not significantly change the output of the model.

Pooling operates independently on every depth slice (channel) of the input and resizes it spatially. The most commonly used types of pooling are max pooling, average pooling, and global pooling.

Max Pooling is the most widely used form of pooling. In this method, a filter (e.g., 2×2) slides over the input feature map, and for each region, the maximum value is taken. This effectively captures the most prominent feature (strongest activation) in that region. Max pooling retains sharp and distinct features and is commonly used after convolution and activation layers.

For example, given a 2×2 region:

$$\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$$

Max pooling outputs: 4

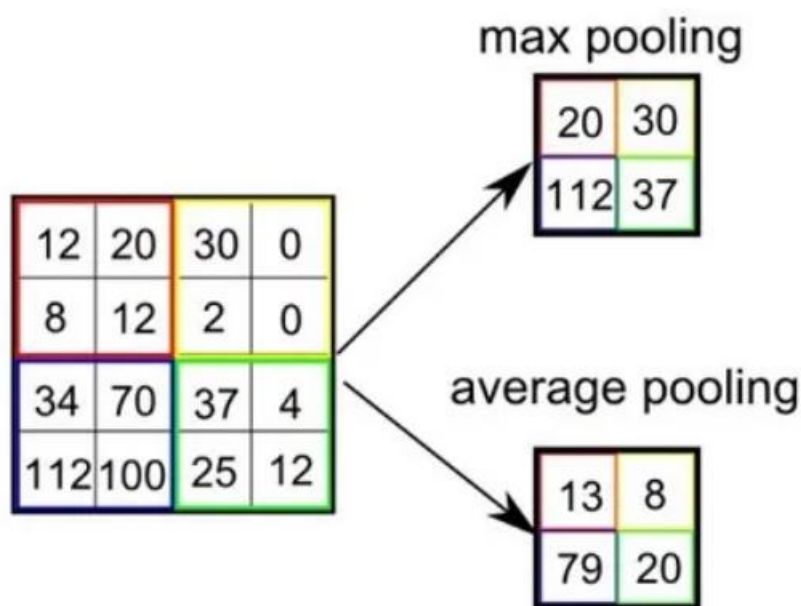
Average Pooling computes the average of the values in each region covered by the filter. It is less aggressive than max pooling and provides smoother representations. Average pooling may be useful in certain applications where subtle patterns and overall trends are more important than the strongest signals.

- Max Pooling- calculates maximum of the each block of feature map.
- Average Pooling- calculates average of the each block of feature map.
- Min Pooling- calculates minimum of the each block of feature map.

Why Max Pooling is used?

Two main reasons why max pooling is effective is :

- It reduces the amount of parameters going forward and hence computational load.
- Higher valued pixels are the most activated and hence captured in this operation.



Max pooling and Average Pooling comparison

Max Pooling Process

Since we have got a basic idea what max pooling intends to do, let's discuss about the operation what it actually does.

We have discussed about role of kernels in the previous post. They act as filters and when they convolve over an image, they create an output corresponding to computations of the filter applied.

For max pooling, we define

- filter(or kernel) of size $n \times n$
- stride value k (by how many pixels we want our filter to move),

For each movement of the filter from the $n \times n$ pixels block of the image under consideration at that point, maximum value is captured as the output for the next layer. Then the filter is moved by k pixels (defined as stride) to perform the same operation again. One key point to be noted that Max pooling is applied after a convolutional layer.



Max pooling using kernel of size 2×2 with stride 2

Global Pooling is a special case of pooling where the pooling operation is applied over the entire feature map instead of small regions. In global max pooling, the maximum value from the entire map is chosen, and in global average pooling, the average of all values is computed. These are often used before the final output layer to convert feature maps into a single vector, especially in architectures designed for image classification.

In addition to the type of pooling, the choice of stride (the number of pixels the filter moves) and whether or not to use padding (adding zeros around the input) also affects the output size and information retained. A stride of 2 is common and reduces the size of feature maps by half in both dimensions.

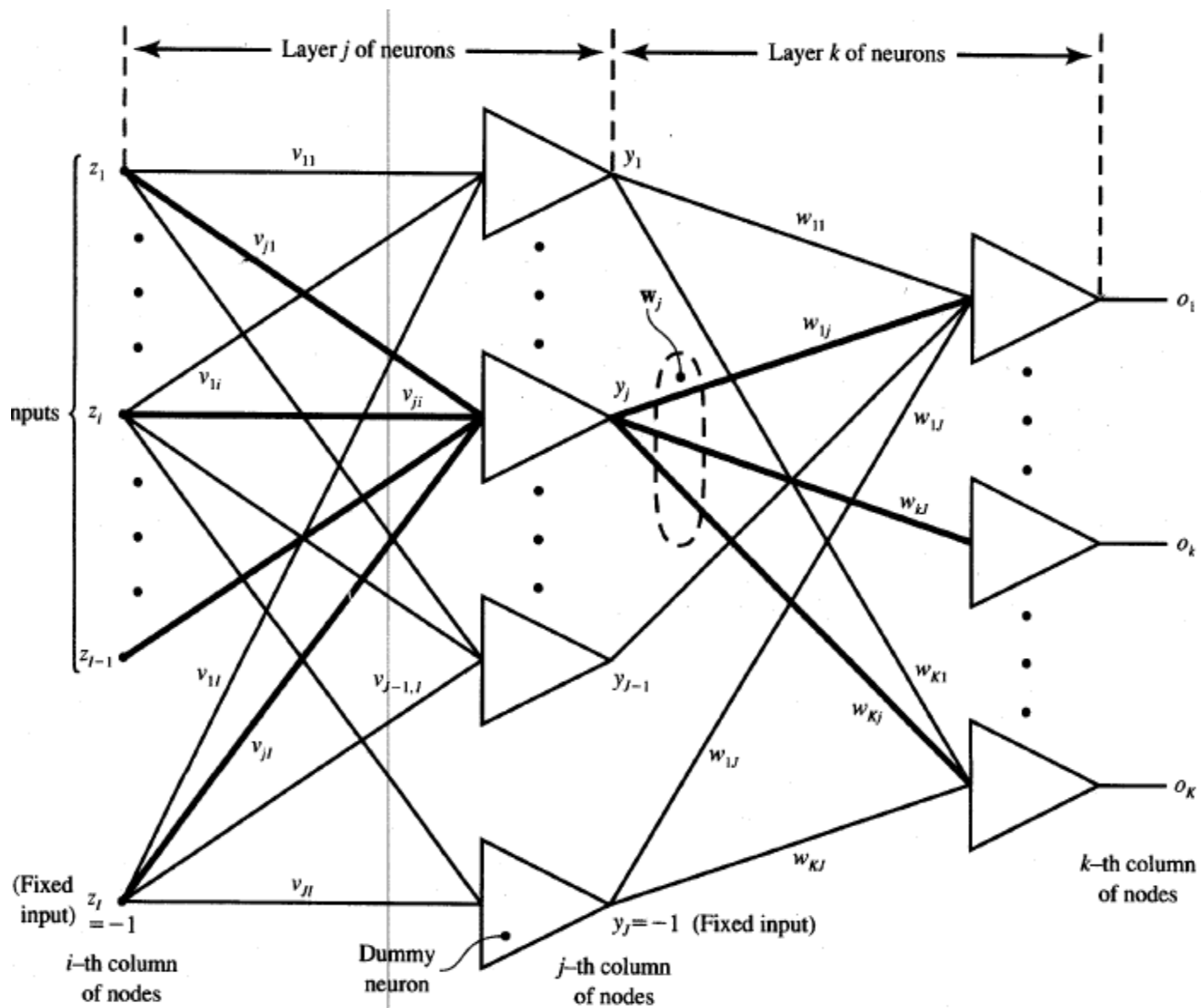
While pooling is extremely useful, it also leads to loss of spatial information. In recent architectures, alternatives like strided convolutions and attention mechanisms are being explored to retain more detailed spatial information.

In conclusion, pooling is an essential technique in CNNs for dimensionality reduction, improved generalization, and computational efficiency. The choice between max, average, or global pooling depends on the specific task and the desired behavior of the model.

Training of CNN in TensorFlow

Training a Convolutional Neural Network (CNN) using TensorFlow, an open-source deep learning framework developed by Google, is one of the most efficient and scalable ways to build and deploy

ERROR BACKPROPAGATION TRAINING



Layered feed forward network with hidden and output layers

EBPTA

Given are P training pairs

$$\{\mathbf{z}_1, \mathbf{d}_1, \mathbf{z}_2, \mathbf{d}_2, \dots, \mathbf{z}_P, \mathbf{d}_P\},$$

where $\mathbf{z}_i$ is $(I \times 1)$, $\mathbf{d}_i$ is $(K \times 1)$, and $i = 1, 2, \dots, P$. Note that the I 'th component of each $\mathbf{z}_i$ is of value -1 since input vectors have been augmented. Size $J - 1$ of the hidden layer having outputs $\mathbf{y}$ is

selected. Note that the J 'th component of $\mathbf{y}$ is of value -1 , since hidden layer outputs have also been augmented; $\mathbf{y}$ is $(J \times 1)$ and $\mathbf{o}$ is $(K \times 1)$.

EBPTA

Step 1: $\eta > 0$, $E_{\max}$ chosen.

Weights $\mathbf{W}$ and $\mathbf{V}$ are initialized at small random values; $\mathbf{W}$ is $(K \times J)$, $\mathbf{V}$ is $(J \times I)$.

$$q \leftarrow 1, p \leftarrow 1, E \leftarrow 0$$

Step 2: Training step starts here (See Note 1 at end of list.)

Input is presented and the layers' outputs computed [$f(\text{net})$ as in (2.3a) is used]:

$$\mathbf{z} \leftarrow \mathbf{z}_p, \mathbf{d} \leftarrow \mathbf{d}_p$$

$$y_j \leftarrow f(\mathbf{v}_j^t \mathbf{z}), \quad \text{for } j = 1, 2, \dots, J$$

where $\mathbf{v}_j$, a column vector, is the j 'th row of $\mathbf{V}$, and

$$o_k \leftarrow f(\mathbf{w}_k^t \mathbf{y}), \quad \text{for } k = 1, 2, \dots, K$$

where $\mathbf{w}_k$, a column vector, is the k 'th row of $\mathbf{W}$.

Step 3: Error value is computed:

$$E \leftarrow \frac{1}{2}(d_k - o_k)^2 + E, \quad \text{for } k = 1, 2, \dots, K$$

Step 4: Error signal vectors δ_o and δ_y of both layers are computed.

Vector δ_o is $(K \times 1)$, δ_y is $(J \times 1)$. (See Note 2 at end of list.)

The error signal terms of the output layer in this step are

$$\delta_{ok} = \frac{1}{2}(d_k - o_k)(1 - o_k^2), \quad \text{for } k = 1, 2, \dots, K$$

The error signal terms of the hidden layer in this step are

$$\delta_{yj} = \frac{1}{2}(1 - y_j^2) \sum_{k=1}^K \delta_{ok} w_{kj}, \quad \text{for } j = 1, 2, \dots, J$$

Step 5: Output layer weights are adjusted:

$$w_{kj} \leftarrow w_{kj} + \eta \delta_{ok} y_j, \quad \text{for } k = 1, 2, \dots, K \text{ and} \\ j = 1, 2, \dots, J$$

Step 6: Hidden layer weights are adjusted:

$$v_{ji} \leftarrow v_{ji} + \eta \delta_{yj} z_i, \quad \text{for } j = 1, 2, \dots, J \text{ and} \\ i = 1, 2, \dots, I$$

Step 7: If $p < P$ then $p \leftarrow p + 1$, $q \leftarrow q + 1$, and go to Step 2;
otherwise, go to Step 8.

EBPTA

Step 8: The training cycle is completed.

For $E < E_{\max}$ terminate the training session. Output weights W , V , q , and E .

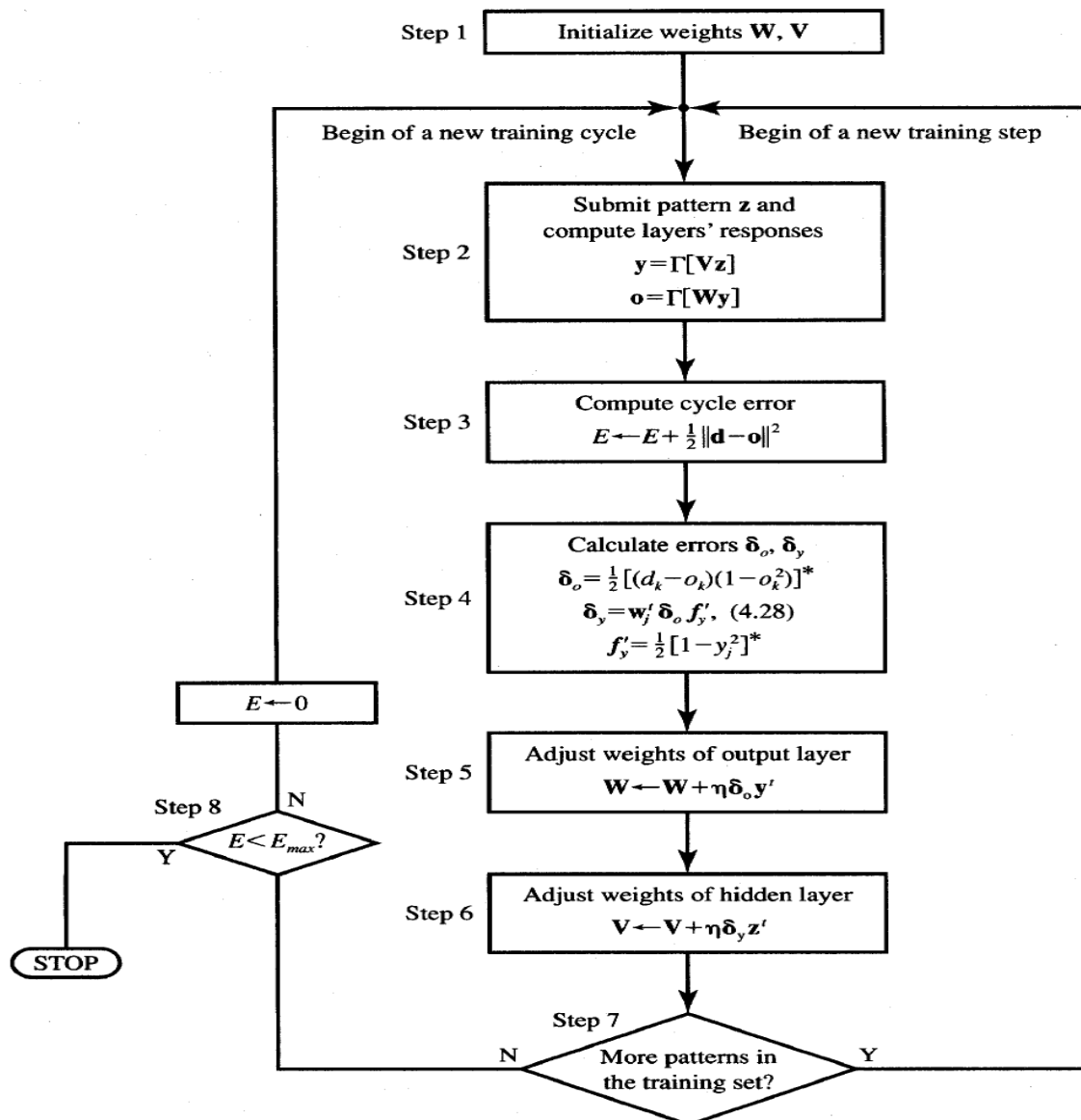
If $E > E_{\max}$, then $E \leftarrow 0$, $p \leftarrow 1$, and initiate the new training cycle by going to Step 2.

■ NOTE 1 For best results, patterns should be chosen at random from the training set (justification follows in Section 4.5).

■ NOTE 2 If formula (2.4a) is used in Step 2, then the error signal terms in Step 4 are computed as follows

$$\delta_{ok} = (d_k - o_k)(1 - o_k)o_k, \quad \text{for } k = 1, 2, \dots, K$$

$$\delta_{yj} = y_j(1 - y_j) \sum_{k=1}^K \delta_{ok} w_{kj}, \quad \text{for } j = 1, 2, \dots, J$$



*If $f(net)$ given by (2.4a) is used in Step 2, then in Step 4 use
 $\delta_o = [(d_k - o_k)(1 - o_k)o_k]$, $f'_y = [(1 - y_j)y_j]$

(a)

Figure 4.8a Error back-propagation training (EBPT algorithm): (a) algorithm flowchart.

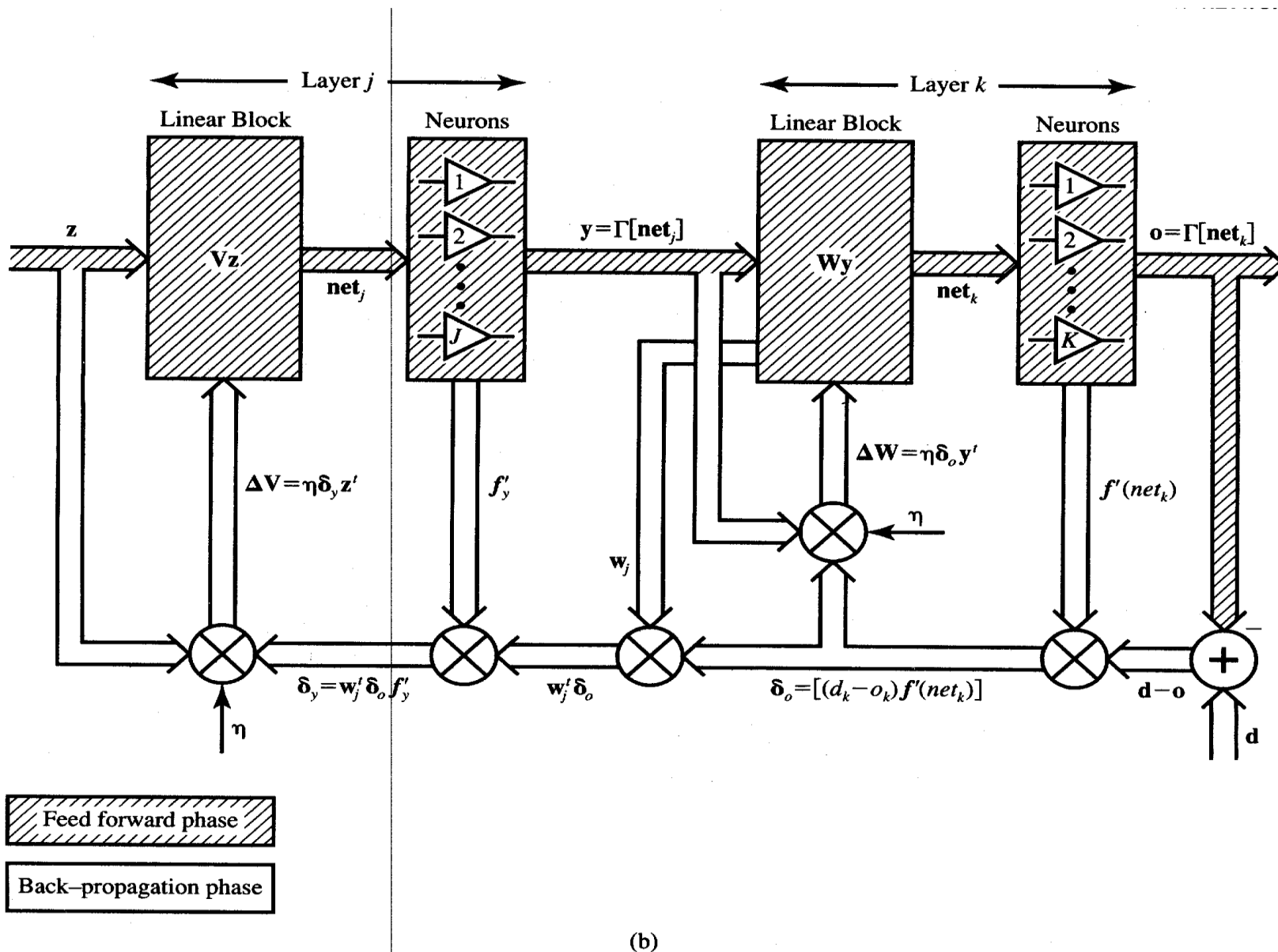


Figure 4.8b Error back-propagation training (EBPT algorithm) (*continued*): (b) block diagram illustrating forward and backward signal flow.

Learning Factors

Initial Weights

- Initialization strongly affects the ultimate solution
- Initial weights with equal weight values → if the solution requires unequal weights to be developed → wrong training
- Network disturbance by random factors or the random character of input patterns during training → result in symmetric weights
- Continuing training beyond a certain low-level results in the undesirable drift of weights-
 - network learning should be restarted with other random weights

Cumulative weight Adjustment versus incremental Updating

- Cumulative weight adjustment after each completed training cycle
- Advisable to use the incremental weight updating after each pattern presentation, but choose patterns in a random sequence from a training set

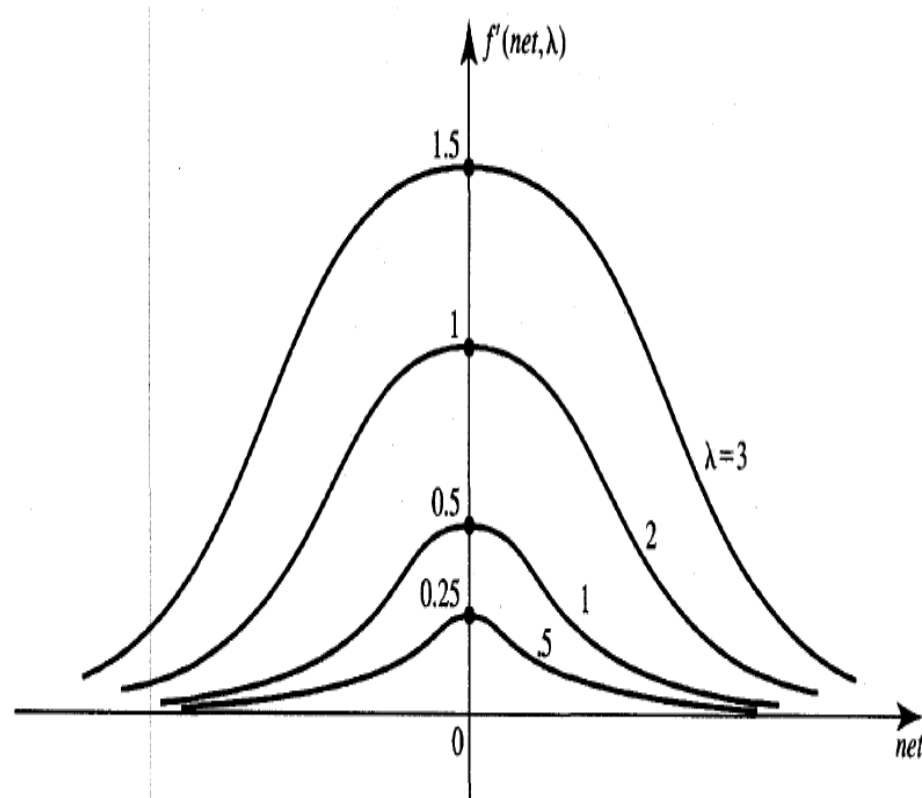
Steepness of the Activation Function (Steepness factor)

- Determines slope of activation function
- Depends on value of net
- multiplying factor in building components of the error signal vectors δ_o , and δ_y
- both the choice and shape of the activation function strongly affect the speed of network learning
- derivative of the activation function-

$$f'(net) = \frac{2\lambda \exp(-\lambda net)}{[1 + \exp(-\lambda net)]^2}$$

- reaches a maximum value of $1/2\lambda$ at $net = 0$

Steepness of the Activation Function



- For a fixed learning constant all adjustments of weights in proportion to the steepness coefficient λ
- Using activation functions with large λ may yield results similar as in the case of large learning constant
- Keep λ at a standard value of 1, and control the learning speed using solely the learning constant η , rather than controlling both λ and η

Learning Constant

- choice of the learning constant $\rightarrow$ depends strongly on the class of the learning problem and on the network architecture
- values ranging from 10^{-3} to 10 reported as successful
- For large learning constants, the learning speed can be drastically increased

Momentum Method (Factor)

- To accelerate the convergence of the error back-propagation learning algorithm

$$\Delta w(t) = -\eta \nabla E(t) + \alpha \Delta w(t-1) \quad \leftarrow \text{momentum term}$$

- α is chosen between 0.1 and 0.8

Introduction to Artificial Neural Networks (ANN),

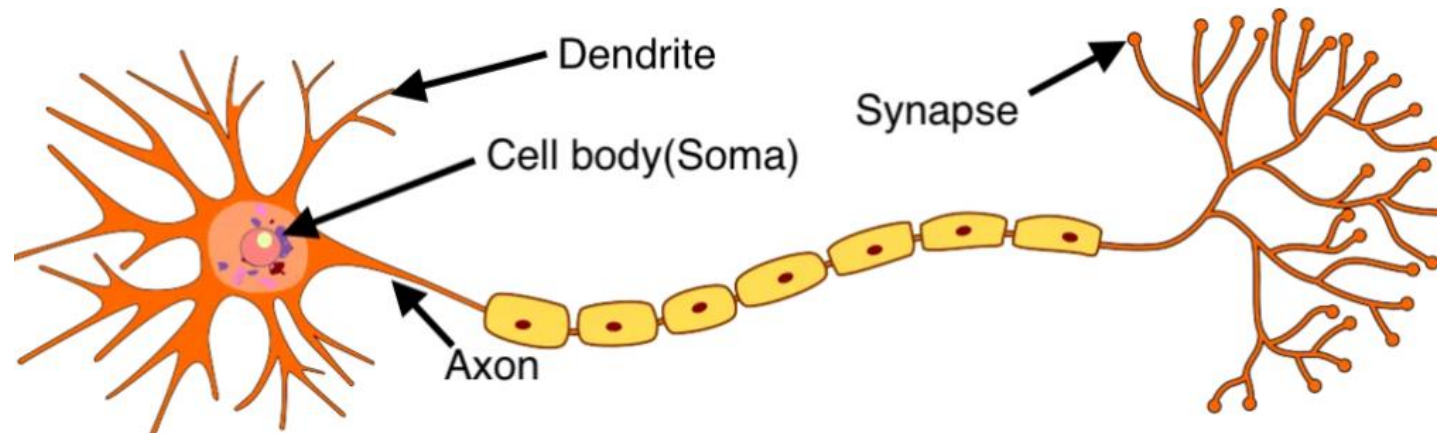
Neural Network

- **Neural network:** *information processing paradigm inspired by biological nervous systems, such as our brain.*
- Structure: large number of highly interconnected processing elements (*neurons*) working together
- *Like people, they learn from experience (by example) It is configured for special application such as pattern recognition and data classification through a learning process.*

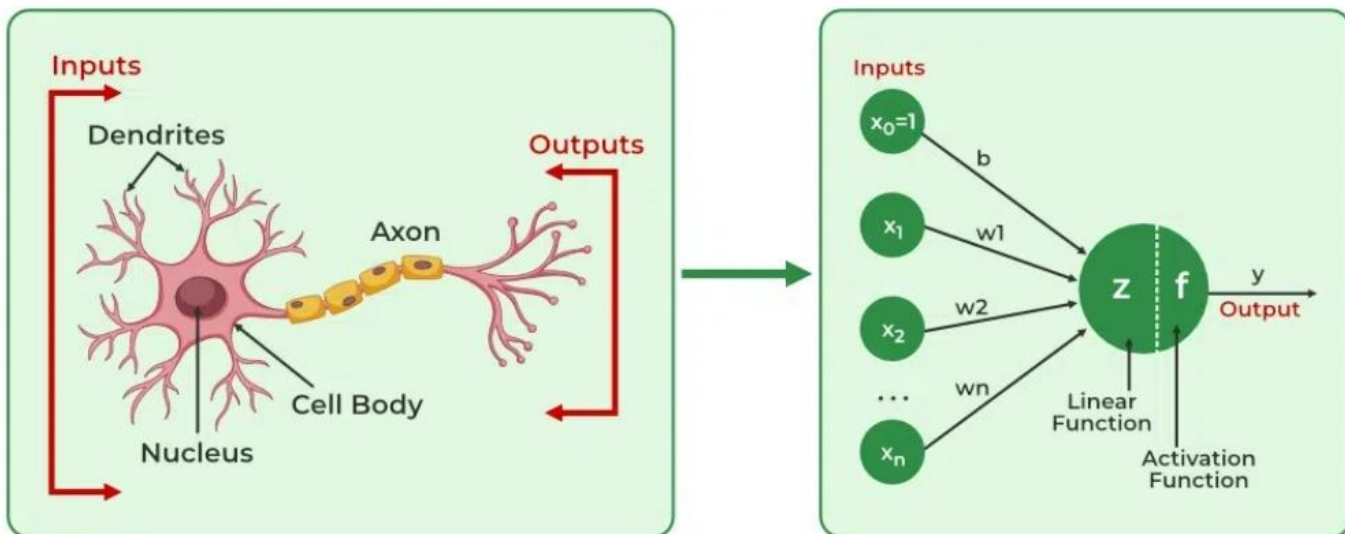
Artificial Neurons

• ANN is an information processing system that has certain performance characteristics in common with biological nets.

1. *The processing element receives many signals.*
2. *Signals may be modified by a weight at the receiving synapse.*
3. *The processing element sums the weighted inputs.*
4. *Under appropriate circumstances (sufficient input), the neuron transmits a single output.*
5. *The output from a particular neuron may go to many other neurons.*

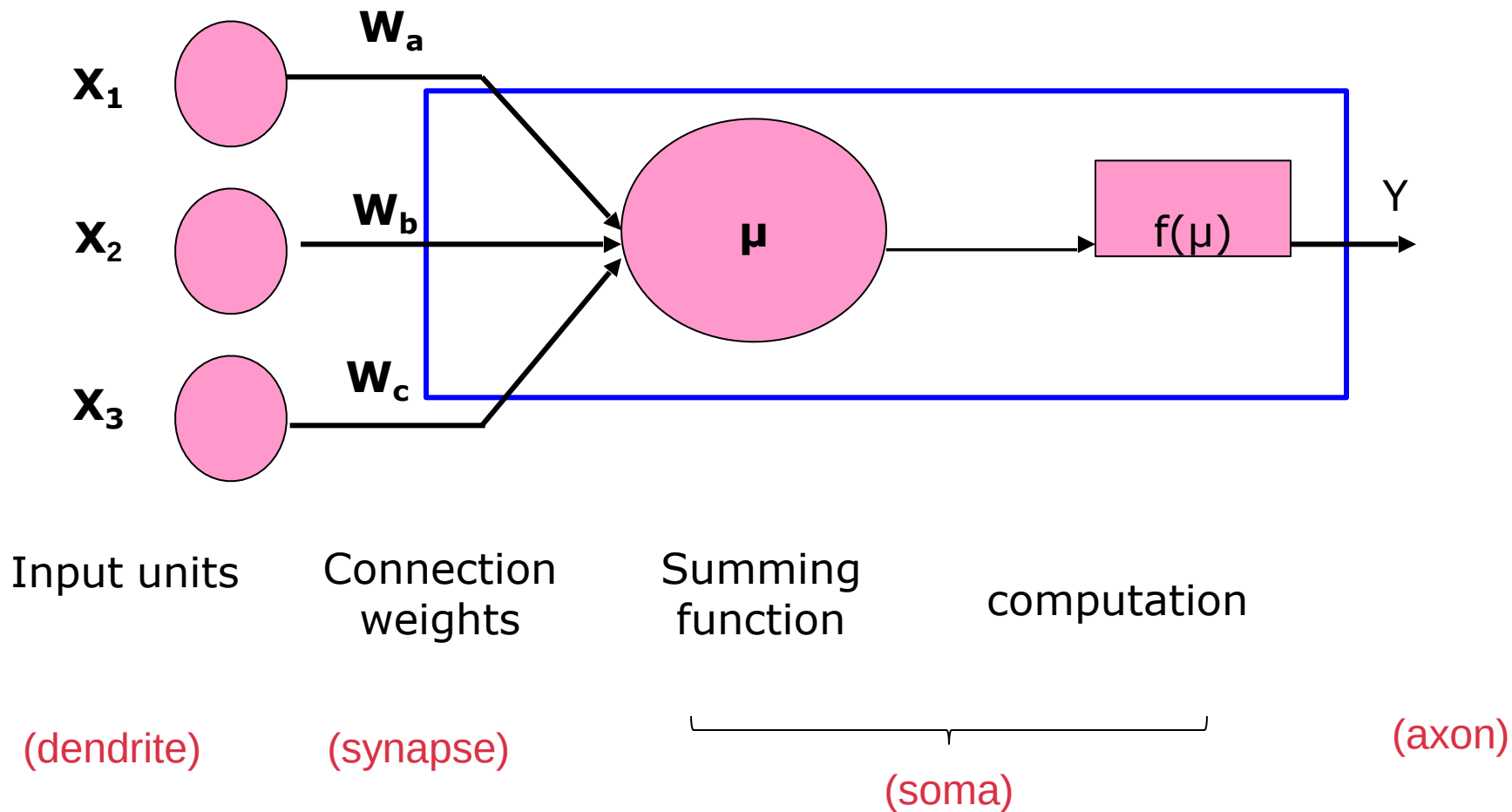


Terminology Relation Between Biological And Artificial Neuron



Biological Neuron	Artificial Neuron
Cell	Neuron
Dendrites	Weights or interconnections
Soma	Net input
Axon	Output

Model Of A Neuron



Model Of A Neuron

- A neural net consists of a large number of simple processing elements called neurons, units, cells or nodes.
- Each neuron is connected to other neurons by means of directed communication links, each with associated weight.
- The weight represent information being used by the net to solve a problem.

Model Of A Neuron

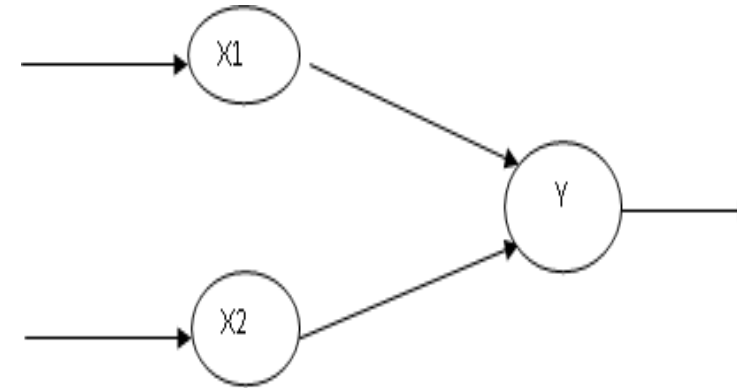
- Each neuron has an internal state, called its activation or activity level, which is a function of the inputs it has received. Typically, a neuron sends its activation as a signal to several other neurons.
- It is important to note that a neuron can send only one signal at a time, although that signal is broadcast to several other neurons.

Application Scope of Neural Networks

- Air traffic control
- Animal behavior
- Appraisal and valuation of property, etc.,
- Betting on horse races, stock markets
- Criminal sentencing
- Complex physical and chemical process
- Data mining, cleaning and validation
- Direct mail advertisers
- Echo patterns
- Economic modeling
- Employee hiring
- Expert consultants
- Fraud detection
- Hand writing and typewriting
- Lake water levels
- Machinery controls
- Medical diagnosis
- Music composition
- Photos and finger prints
- Recipes and chemical formulation
- Traffic flows
- Weather prediction

Basic Operation of a Neural Net

- X1 and X2 – input neurons.
- Y- output neuron
- Weighted interconnection links- W1 and W2.
- Net input calculation is :



$$Y_{in} = x_1w_1 + x_2w_2 + \dots + x_nw_n$$

$$\sum_{i=1}^n x_iw_i$$

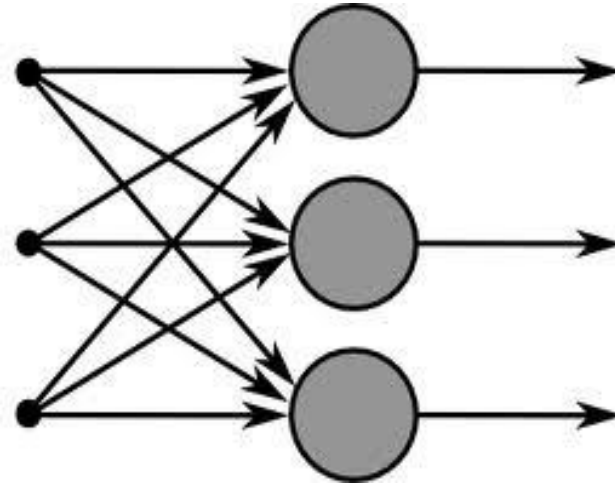
- Output is : $y = f(Y_{in})$

Basic models of ANN

- Models are based on three entities
 - *The model's synaptic interconnections.*
 - *The training or learning rules adopted for updating and adjusting the connection weights.*
 - *Their activation functions*
- The arrangement of neurons to form layers and the connection pattern formed within and between layers is called the **network architecture**.
- Five types:
 - *Single layer feed forward network*
 - *Multilayer feed-forward network*
 - *Single node with its own feedback*
 - *Single-layer recurrent network*
 - *Multilayer recurrent network*

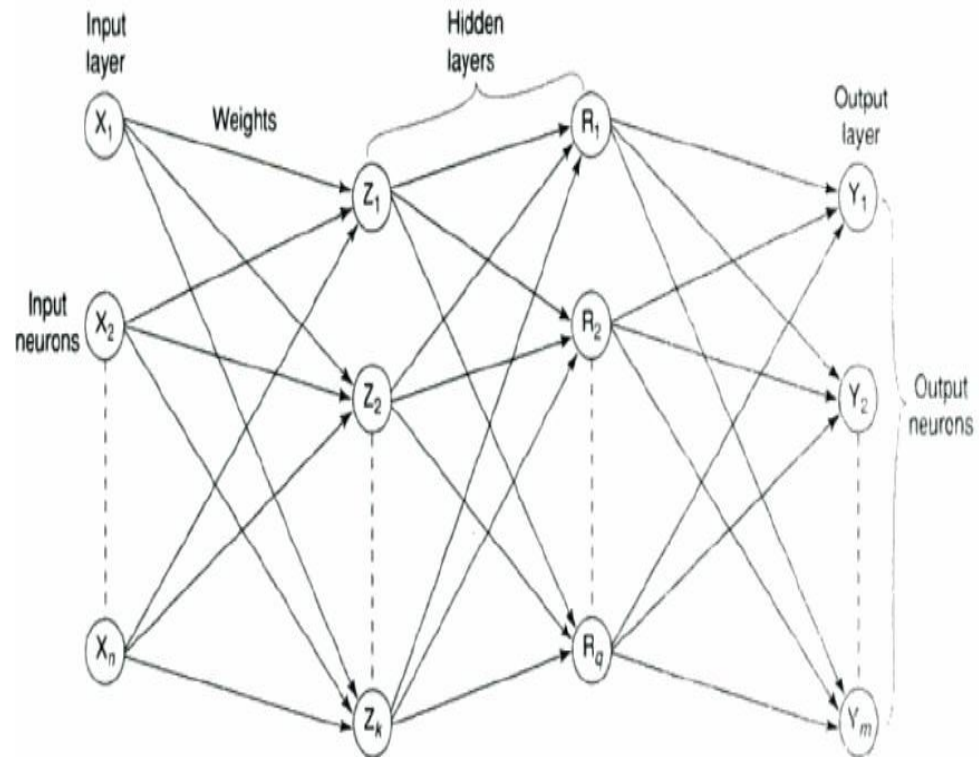
Single layer Feed-Forward Network

- Layer is formed by taking processing elements and combining it with other processing elements.
- Input and output are linked with each other
- Inputs are connected to the processing nodes with various weights, resulting in series of outputs one per node.



Multilayer feed-forward network

- Formed by the interconnection of several layers.
- Input layer receives input and buffers input signal.
- Output layer generated output.
- Layer between input and output is called *hidden layer*.
- Hidden layer is internal to the network.
- Zero to several hidden layers in a network.
- More the hidden layer, more is the complexity of network, but efficient output is produced.



Feed back network

- If no neuron in the output layer is an input to a node in the same layer / proceeding layer – **feed forward network**.
- If outputs are directed back as input to the processing elements in the same layer/proceeding layer – **feedback network**.
- If the output are directed back to the input of the same layer then it is **lateral feedback**.
- **Recurrent networks** are networks with feedback networks with closed loop.
- Fig 2.8 (A) –simple recurrent neural network having a single neuron with feedback to itself.
- Fig 2.9 – single layer network with feedback from output can be directed to processing element itself or to other processing element/both.

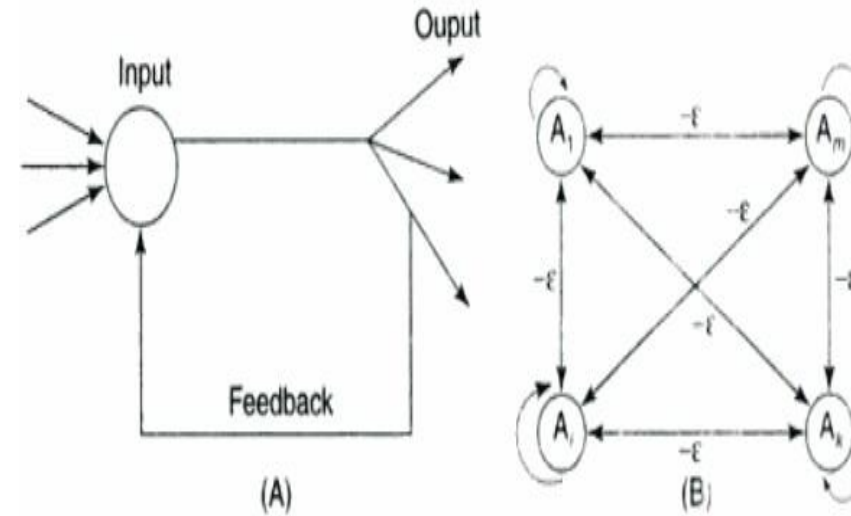


Figure 2-8 (A) Single node with own feedback. (B) Competitive nets.

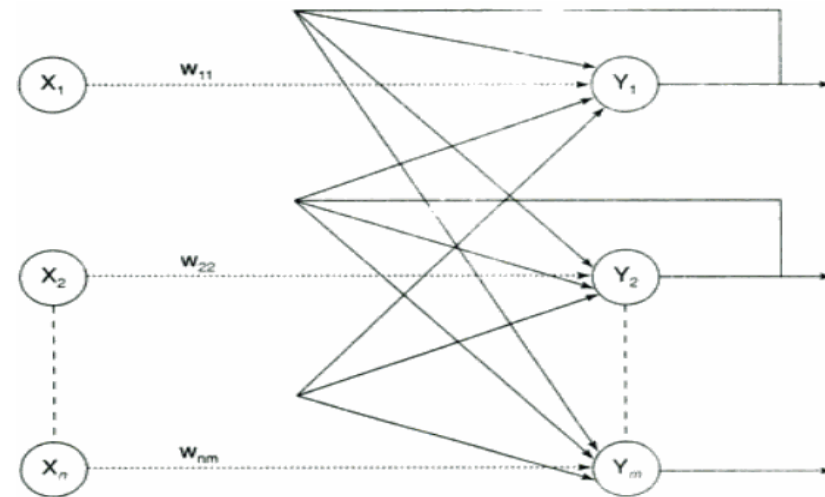
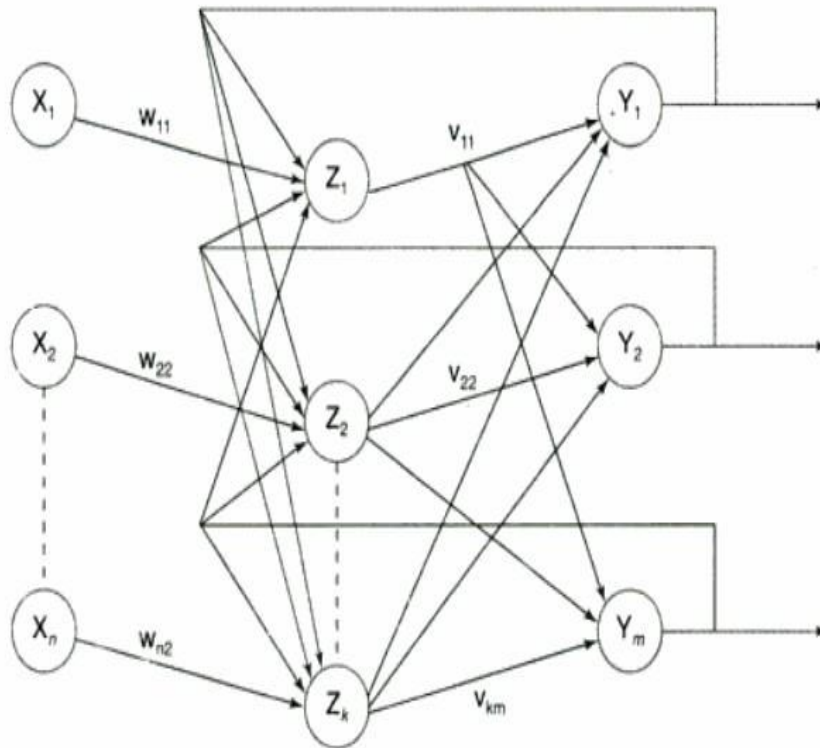


Figure 2-9 Single-layer recurrent network.



- Processing element output can be directed back to the nodes in the preceding layer, forming a ***multilayer recurrent network***.
- Processing element output can be directed to processing element itself or to other processing element in the same layer.

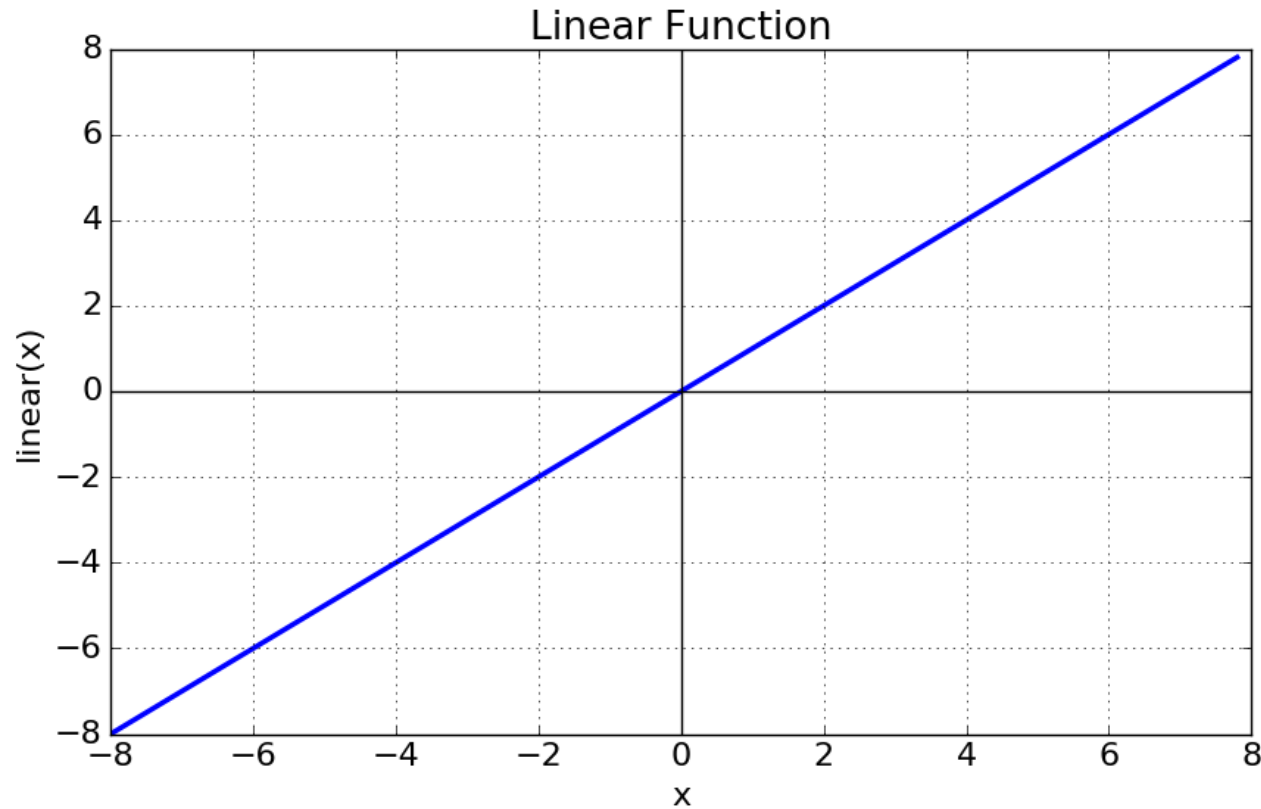
Activation Function

Activation function is used to determine the output of neural network like yes or no. It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).

Types:

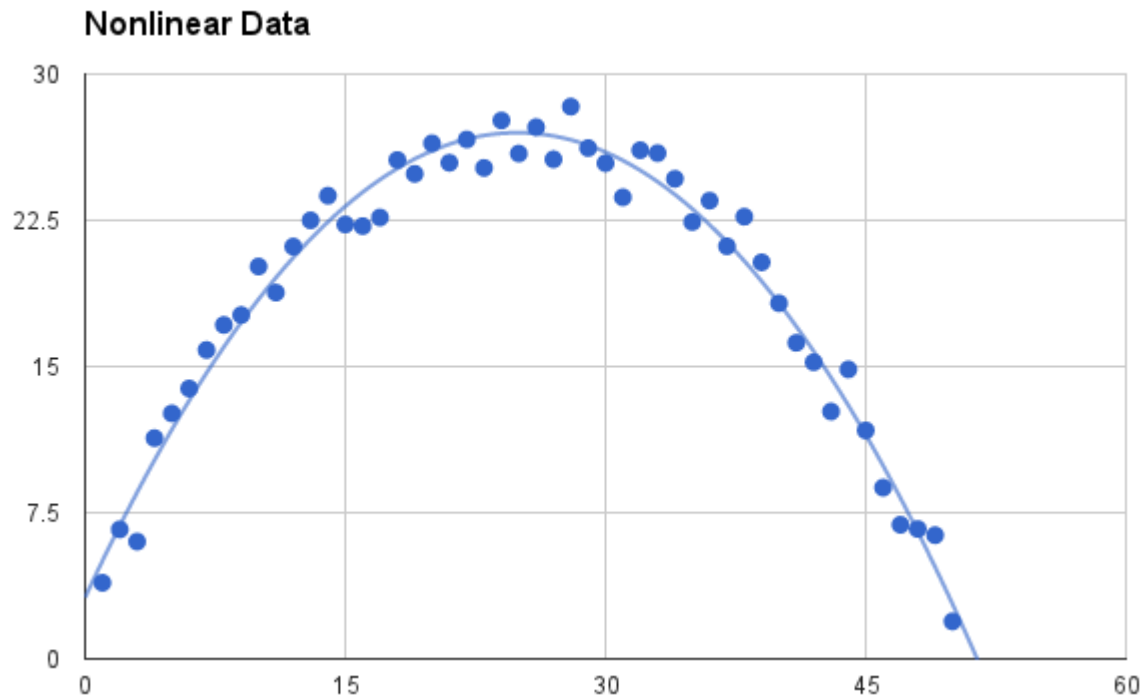
- ◎ Linear Activation Function
- ◎ Non-linear Activation Functions

Linear or Identity Activation Function



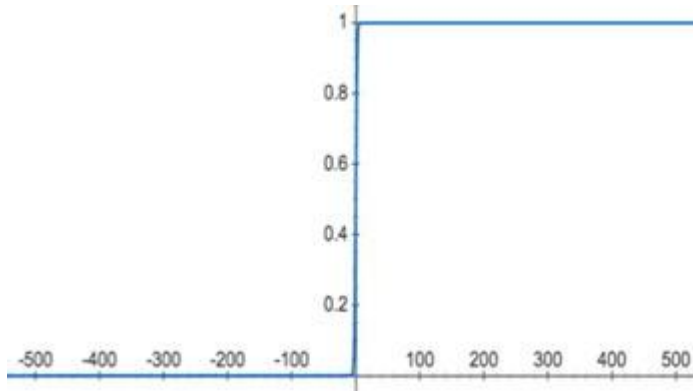
- **Equation :** $f(x) = x$
- **Range :** $-\infty$ to $+\infty$
- It doesn't help with the complexity or various parameters of usual data that is fed to the neural networks.

Non-linear Activation Function



- **Derivative or Differential:** Change in y-axis w.r.t. change in x-axis. It is also known as slope.
- **Monotonic function:** A function which is either entirely non-increasing or non-decreasing.

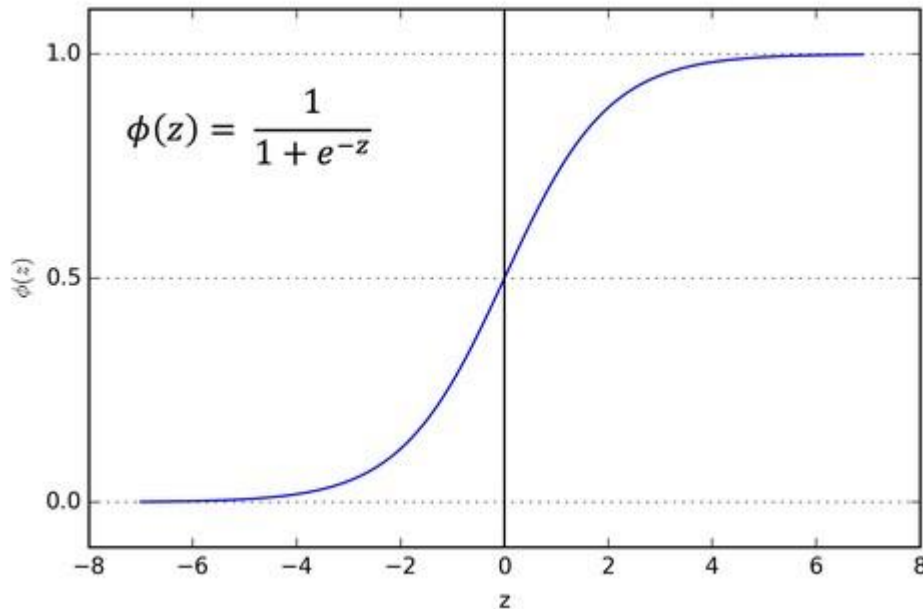
Binary Step Function



$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{For } x \geq 0 \end{cases}$$

- Useful for binary classifier
- Not useful for multi-class classification problem.
- Derivative is 0 which means backpropagation is not possible due to the vanishing gradients problem.

Sigmoid or Logistic Activation Function

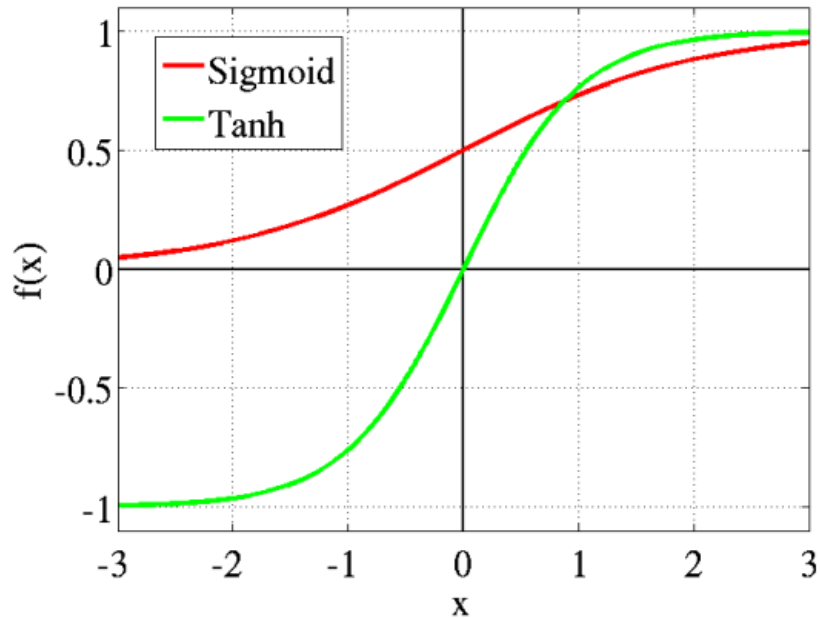


- **Equation** $\phi(z) = \frac{1}{1 + e^{-z}}$
- **Range** : 0 to 1
- It is especially used for models where we have to **predict the probability** as an output.
- **Differentiable** ○ **Monotonic**

The logistic sigmoid function can cause a neural network to get stuck at the training time.

The **softmax function** is a more generalized logistic activation function which is used for multiclass classification.

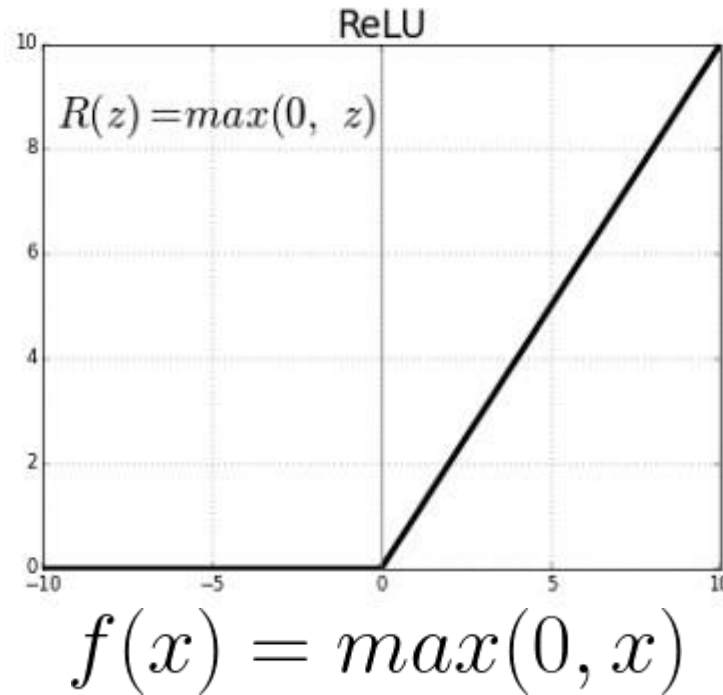
Tanh or hyperbolic tangent Activation Function



$$f(x) = \tanh(x) = \frac{2}{(1 + e^{-2x})} - 1$$

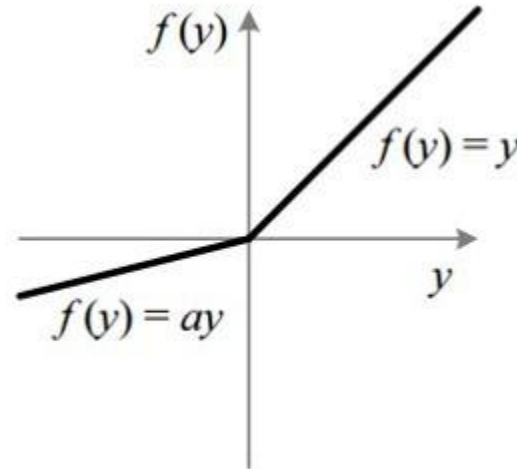
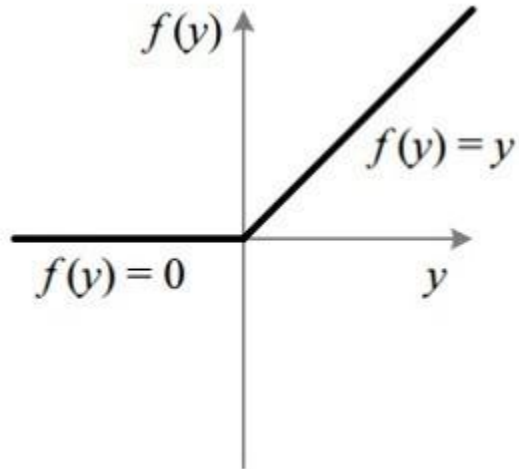
- **'S' shaped like sigmoid**
- **Range : -1 to 1**
- Negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph.
- **Differentiable**
- **Monotonic**
- Used for classification between two classes.
- *Both tanh and logistic sigmoid activation functions are used in feed-forward nets.*

ReLU (Rectified Linear Unit) Activation Function



- ReLU is half rectified (from bottom).
- **Range:** [0 to infinity)
- The function and its derivative **both are monotonic.**
- Widely used in CNN.
- Easy to compute
- Does not saturate
- Does not cause the Vanishing Gradient Problem.
- Not zero centred.
- “**dying ReLU**” problem

Leaky ReLU



- › Solves the dying ReLU problem
- › Usually, the value of **a** is 0.01 or so.
- › When **a is not 0.01** then it is called **Randomized ReLU**.
- › **Range:**
- **Monotonic**



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

